Unidad 7

Callbacks, Promises y Async/Await

Materia: Programación 3 Frontend
Año: 2026
Profesor: Lucas Ferrero

UTN
FACULTAD REGIONAL
RAFAELA

1.Introducción a la Asincronía en JavaScript

¿Por qué existe la asincronía en JS? JavaScript corre en un **único hilo**. Si una tarea tarda (leer un archivo, pedir datos a un servidor), bloquea el programa. La asincronía permite iniciar esas tareas y **seguir ejecutando código del resto del programa** mientras se espera la respuesta de dichas tareas.

```
// ❌ Sin asincronía: la UI se congela hasta que termina
fetch_sincrono("https://api.example.com/datos"); // bloquea todo

// ✅ Con asincronía: el hilo sigue libre
fetch("https://api.example.com/datos").then(...); // no bloquea
```

1.1 ¿Qué es la Programación Síncrona?

La programación síncrona se caracteriza por la **ejecución de tareas de manera secuencial y ordenada**. Esto implica que una tarea debe completarse en su totalidad antes de que la siguiente pueda iniciarse, estableciendo un estilo de ejecución "bloqueante". Las operaciones se realizan una a la vez, siguiendo estrictamente el orden en que fueron escritas en el código.

Una característica distintiva de la programación síncrona es su naturaleza bloqueante: mientras una operación está en curso, cualquier otra instrucción subsiguiente queda en espera, impidiendo su ejecución hasta que la tarea actual finalice.

Para ilustrar este concepto, se puede imaginar una conversación por radio (walkie talkie): una persona habla, y la otra escucha atentamente, esperando a que la primera termine su intervención para poder responder. El flujo de la conversación es estrictamente secuencial.

Considérese el siguiente ejemplo sencillo:

```
console.log("Paso 1: Empezar la tarea.");
// Simula una tarea que toma tiempo, pero es síncrona (ej. un cálculo intensivo de CPU)
for (let i = 0; i < 1000000000; i++) {
  // Tarea pesada de CPU
}
console.log("Paso 2: Tarea pesada completada.");
console.log("Paso 3: Continuar con la siguiente tarea.");
```

En este escenario, el mensaje "Paso 3: Continuar con la siguiente tarea." no se imprimirá en la consola hasta que el bucle for (que simula una tarea computacionalmente intensiva) haya finalizado. Durante la ejecución de este bucle, cualquier otra interacción o actualización de la interfaz de usuario quedaría bloqueada, haciendo que la aplicación parezca no responder.

1.2. ¿Qué es la Programación Asíncrona?

En contraste, la programación asíncrona permite que las tareas se ejecuten de forma independiente, sin que una bloquee el hilo principal de ejecución. Las operaciones que requieren un tiempo considerable, como la recuperación de datos de un servidor o la lectura de archivos, pueden ser iniciadas en segundo plano, permitiendo que otras operaciones continúen su curso de manera simultánea.

Las características clave de la programación asíncrona incluyen su naturaleza no bloqueante, lo que evita que el hilo principal se detenga a la espera de una tarea, mejorando así la capacidad de respuesta de la aplicación. Además, las tareas pueden ejecutarse en cualquier orden o incluso de forma concurrente, lo que optimiza el rendimiento y la eficiencia. Este enfoque es altamente adaptable y **resulta ideal para gestionar múltiples tareas de forma simultánea**, como llamadas a APIs Web o el procesamiento de archivos.

Una analogía común para la programación asíncrona es el proceso de hornear galletas: es posible precalentar el horno y, mientras se espera que alcance la temperatura deseada, comenzar a mezclar los ingredientes. No es necesario esperar a que el horno termine de precalentarse para iniciar la siguiente tarea.

Consideremos el siguiente ejemplo:

```
console.log("Paso 1: Iniciar precalentamiento del horno.");
setTimeout(() => {
  console.log("Paso 3: Horno precalentado (después de 2 segundos).");
}, 2000);
console.log("Paso 2: Mezclar ingredientes.");
// Salida esperada:
// Paso 1: Iniciar precalentamiento del horno.
// Paso 2: Mezclar ingredientes.
// Paso 3: Horno precalentado (después de 2 segundos).
```

En este caso, la función *setTimeout* inicia un temporizador que se ejecuta en segundo plano. El código subsiguiente ("Paso 2: Mezclar ingredientes.") no se bloquea y se ejecuta inmediatamente. El mensaje del "Paso 3" aparecerá en la consola solo después de que transcurran los 2 segundos especificados, demostrando el comportamiento no bloqueante de la operación asíncrona.

ELIMINAR

1.3. ¿Por qué JavaScript necesita Asincronía? La Naturaleza de Hilo Único

JavaScript, en su diseño fundamental, es un lenguaje de un solo hilo (single-threaded). Esto significa que el motor de JavaScript solo puede ejecutar una operación a la vez en su hilo principal de ejecución. Esta característica inherente plantea un desafío significativo cuando se trata de operaciones que consumen mucho tiempo.

Si una operación síncrona se prolonga, como una petición de red a un servidor remoto, una lectura de archivo extensa o un cálculo computacionalmente intensivo, bloquearía completamente este único hilo de ejecución. El efecto inmediato de este bloqueo es que la interfaz de usuario (UI) de una aplicación web se congelaría: los botones dejarían de responder, las animaciones se detendrían y la página parecería "colgada". Esta situación es inaceptable en las aplicaciones web modernas, donde la capacidad de respuesta y una experiencia de usuario fluida son primordiales.

Para mitigar este problema y garantizar que la aplicación permanezca interactiva y receptiva, JavaScript recurre a mecanismos asíncronos. Estos mecanismos (como los callbacks, las Promises y async/await) permiten delegar tareas de larga duración a entornos externos, como las Web APIs proporcionadas por el navegador o los módulos C++ en entornos como Node.js.

Una vez que la tarea delegada finaliza en segundo plano, el hilo principal es notificado del resultado, sin haber tenido que esperar activamente su finalización. Esta orquestación es fundamental para que el único hilo de JavaScript pueda manejar operaciones que requieren espera (como las de entrada/salida) de manera no bloqueante, priorizando la capacidad de respuesta de la interfaz de usuario.

Por lo tanto, la asincronía en JavaScript no se trata de una verdadera ejecución paralela en el sentido de múltiples hilos de CPU ejecutando código JavaScript simultáneamente (con la excepción de los Web Workers, que son un caso especial), sino de una sofisticada coordinación que simula la concurrencia para mantener la fluidez de la aplicación.

1.4. El Modelo de Ejecución de JavaScript: Call Stack, Web APIs, Event Loop, Task Queue y Microtask Queue

Para comprender cómo JavaScript gestiona la asincronía a pesar de su naturaleza de hilo único, es esencial desglosar los componentes de su entorno de ejecución. Este modelo permite una "ilusión de concurrencia" que mantiene las aplicaciones responsivas.

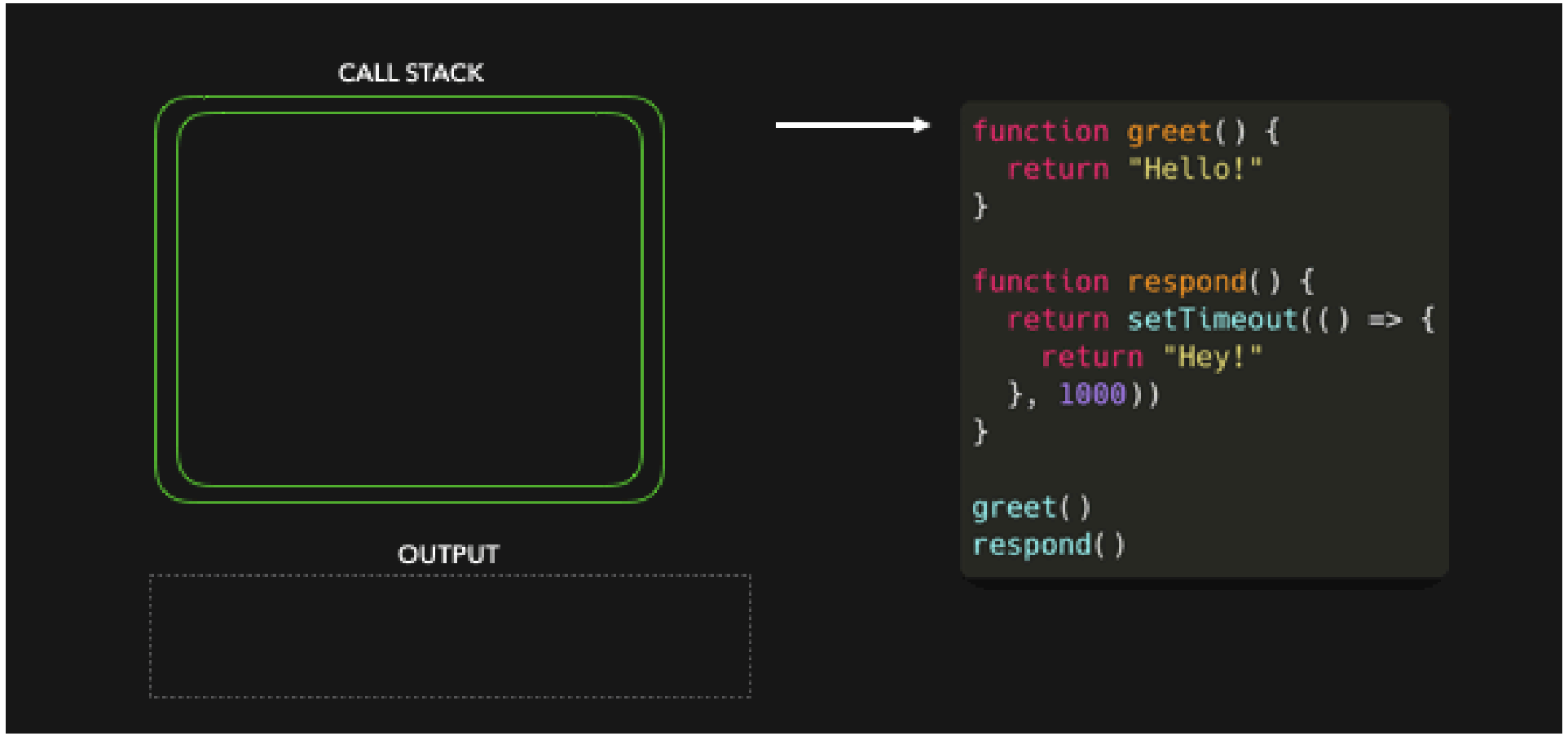
- **Call Stack (Pila de Llamadas):** La Call Stack es una estructura de datos de tipo LIFO (Last-In, First-Out, o "Último en entrar, primero en salir") que registra el contexto de ejecución del programa. Funciona como una "lista de tareas pendientes" para el motor de JavaScript. Cuando una función es invocada, se "empuja" a la parte superior de la pila. Una vez que la función completa su ejecución, se "saca" de la pila. JavaScript procesa las funciones en la pila una por una. El dilema del hilo único se hace evidente aquí: si una función en la pila tarda mucho en ejecutarse, bloquea toda la ejecución subsiguiente hasta que se complete, impidiendo cualquier otra actividad.

- **Web APIs (APIs del Navegador / Entorno):** Las Web APIs son interfaces proporcionadas por el entorno del navegador (o por Node.js en el lado del servidor) que manejan tareas que se encuentran fuera del tiempo de ejecución central de JavaScript. Ejemplos comunes incluyen *setTimeout()* para temporizadores, *fetch()* para peticiones de red, *addEventListener()* para manejar eventos del DOM, y *getUserMedia()* para acceder a la cámara o el micrófono del usuario. Cuando JavaScript encuentra una función asíncrona, la delega a la Web API correspondiente. Esta Web API ejecuta la tarea en un entorno multi-hilo o separado (fuera del hilo principal de JavaScript), liberando así la Call Stack para que pueda procesar otras tareas.

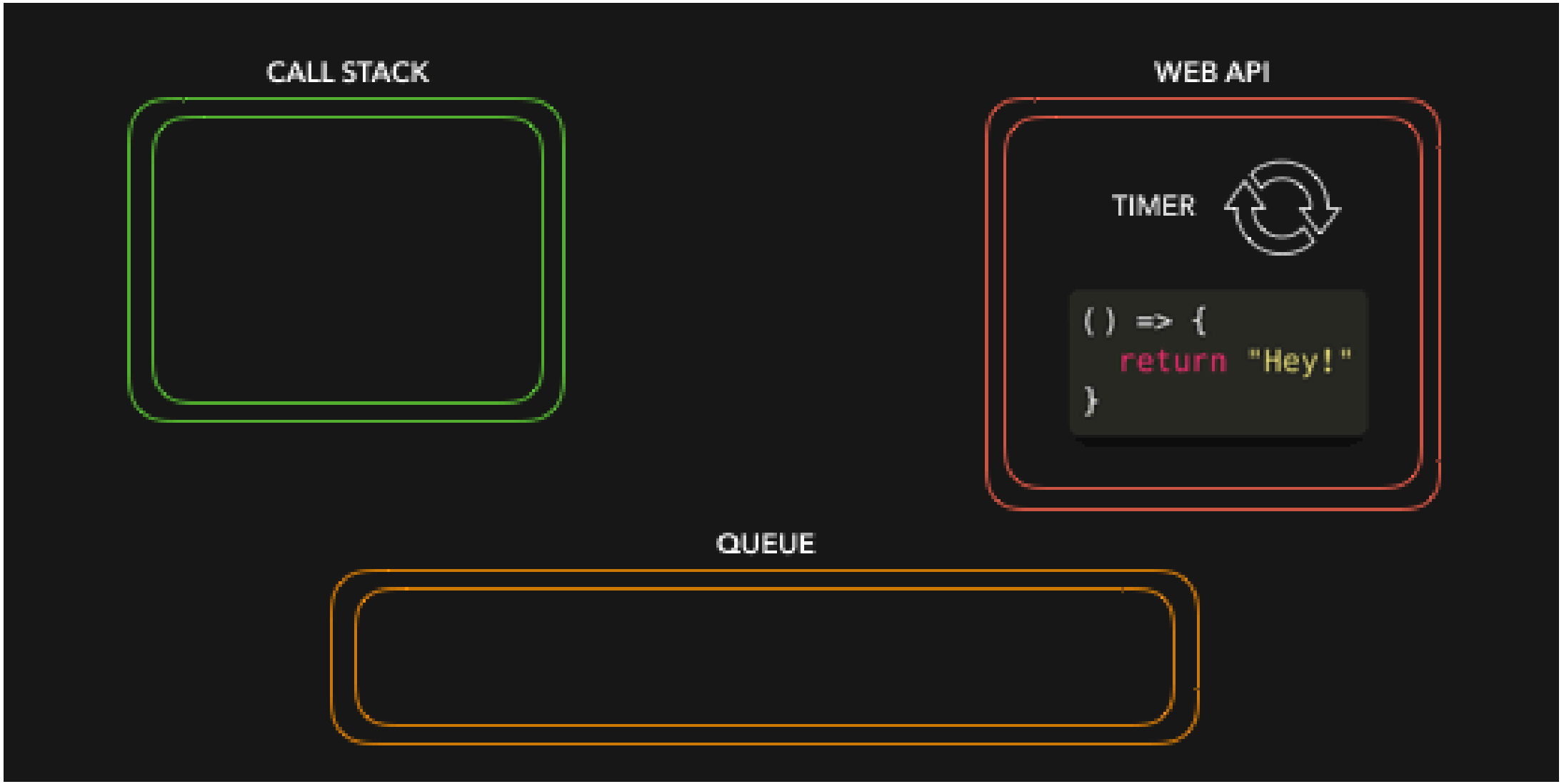
- **Task Queue (Cola de Tareas / Micro Task Queue/ Macro Task Queue)**: La Task Queue es una cola de tipo FIFO (First-In, First-Out, o "Primero en entrar, primero en salir") donde se colocan las funciones de callback una vez que las Web APIs han completado las operaciones que les fueron delegadas. Aquí se encuentran los callbacks de setTimeout(), setInterval(), los manejadores de eventos de usuario (como click o scroll), y las respuestas iniciales de las peticiones de red (como las de fetch).

Event Loop (Bucle de Eventos): El Event Loop es el mecanismo central que orquesta la concurrencia en JavaScript. Su función principal es monitorear continuamente el estado de la Call Stack y de las colas (Macrotask Queue y Microtask Queue). Cuando la Call Stack está vacía, el Event Loop toma una tarea de la Microtask Queue y la mueve a la Call Stack para su ejecución. Una vez que la Microtask Queue está completamente vacía, el Event Loop procede a tomar una tarea de la Macrotask Queue y la mueve a la Call Stack. Este proceso se repite de forma continua. La prioridad de la Microtask Queue es fundamental: el Event Loop siempre la vacía por completo antes de procesar cualquier tarea de la Macrotask Queue.

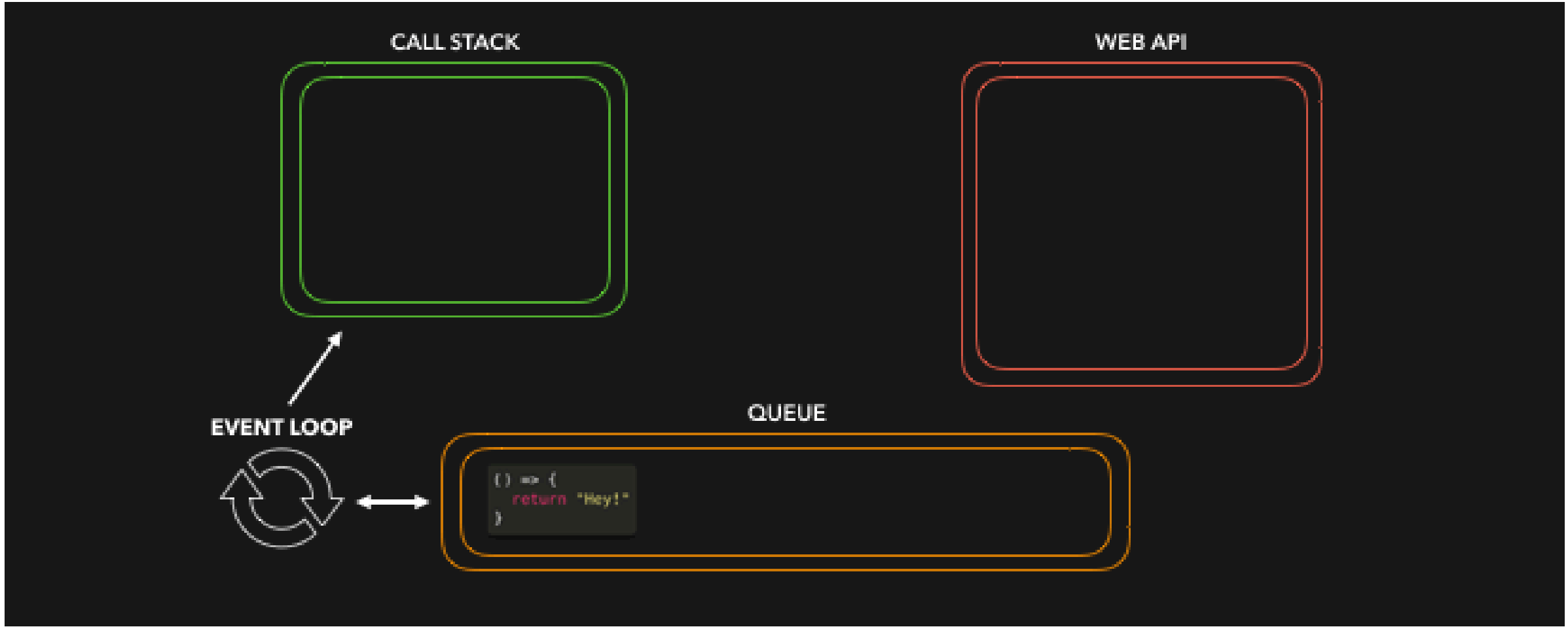
Ejemplos visuales del modelo



Flujo de la Callback del setTimeout

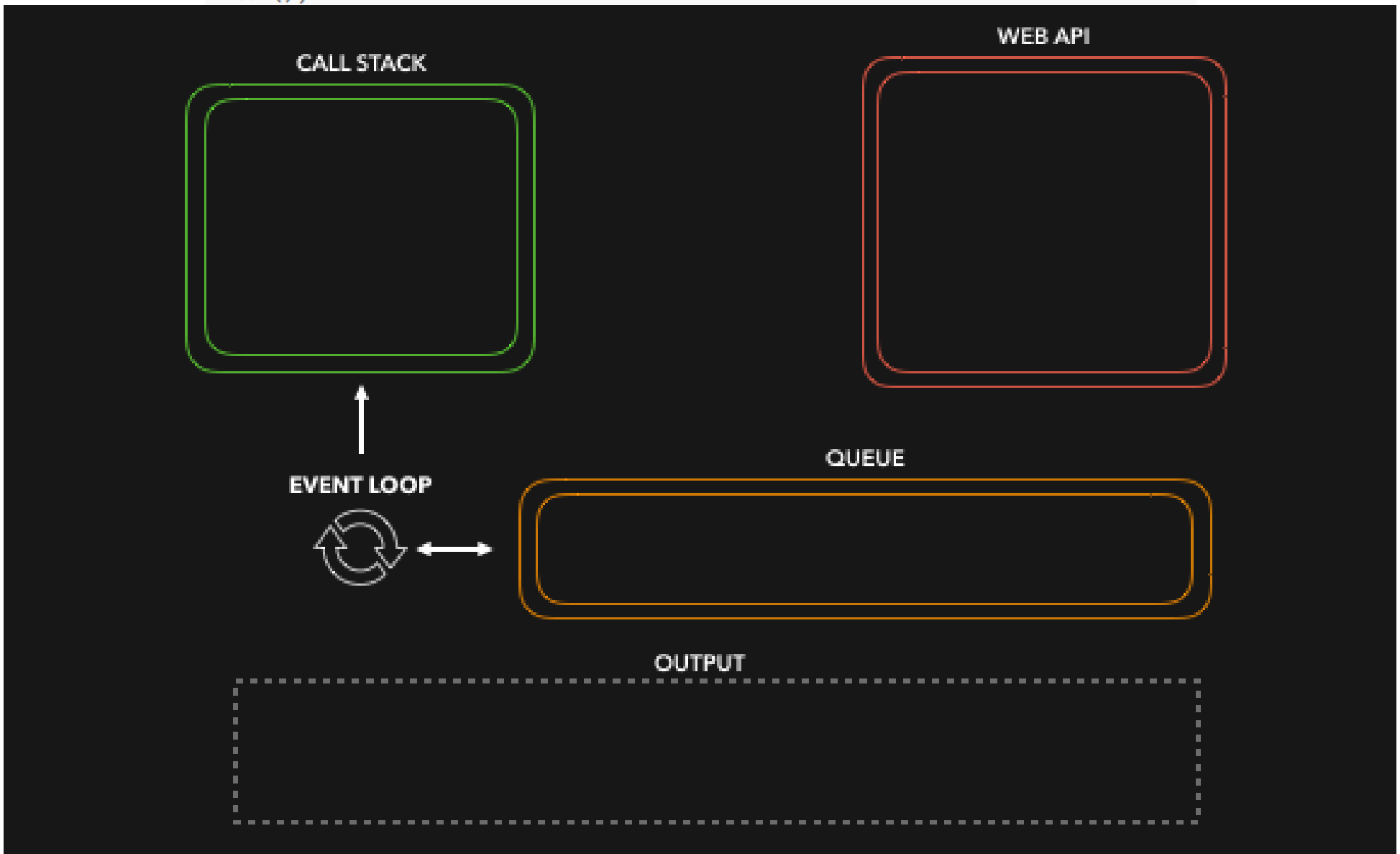


La callback es empujada al CallStack por el EventLoop

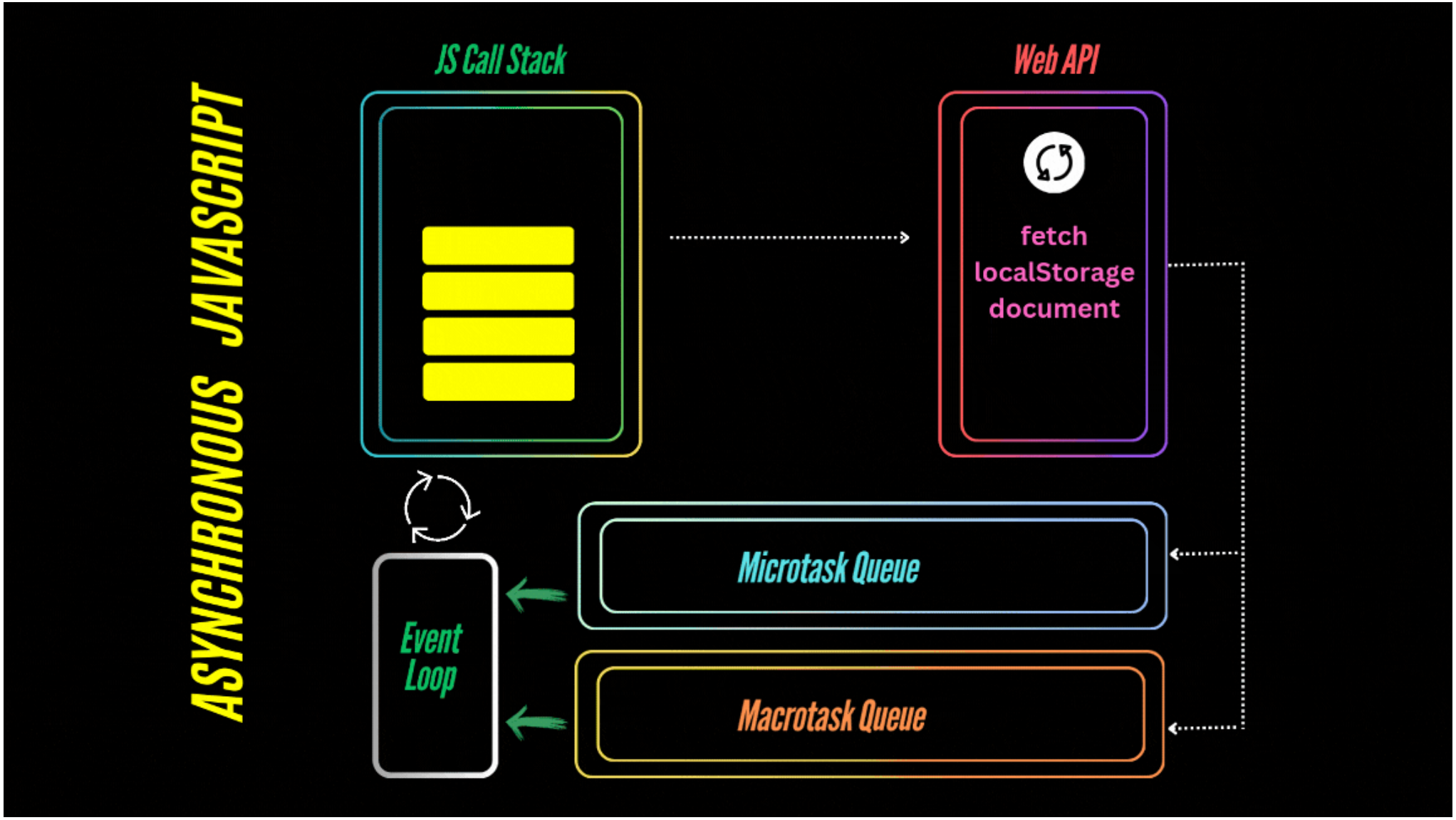


Otro ejemplo

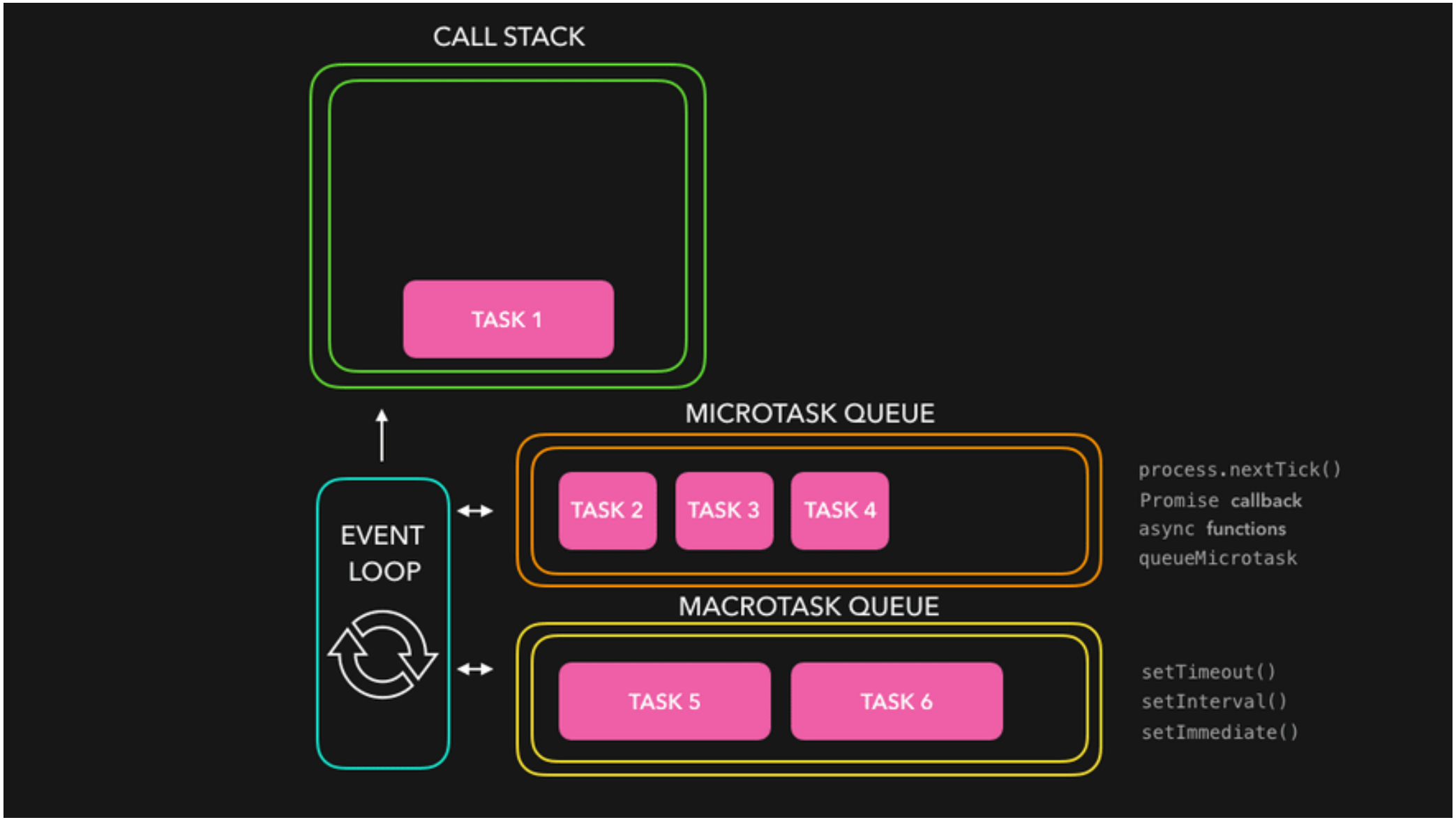
```
const foo = () => console.log("First");
const bar = () => setTimeout(() => console.log("Second"), 500);
const baz = () => console.log("Third");
bar();
foo();
baz();
```



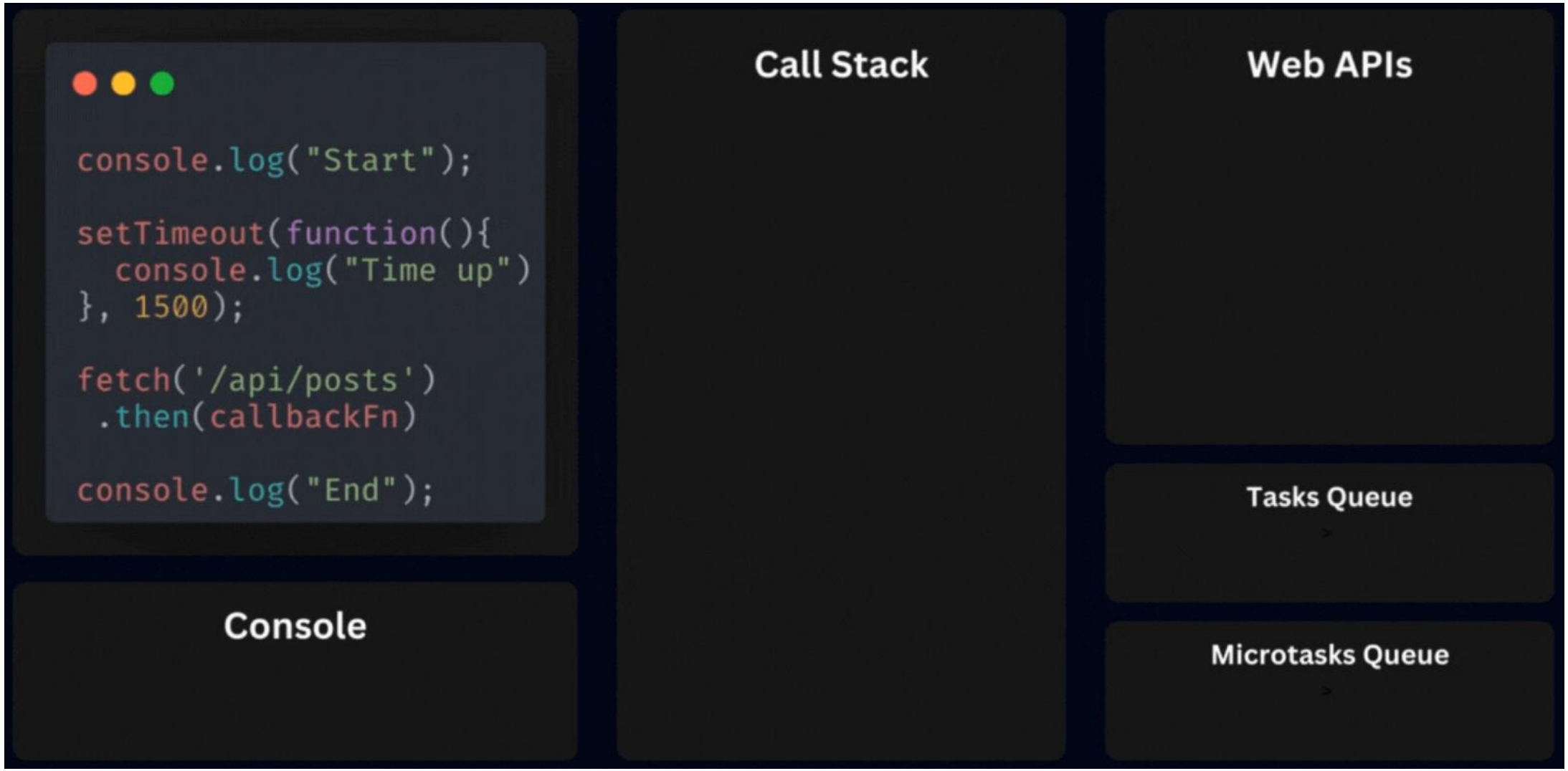
Modelo completo

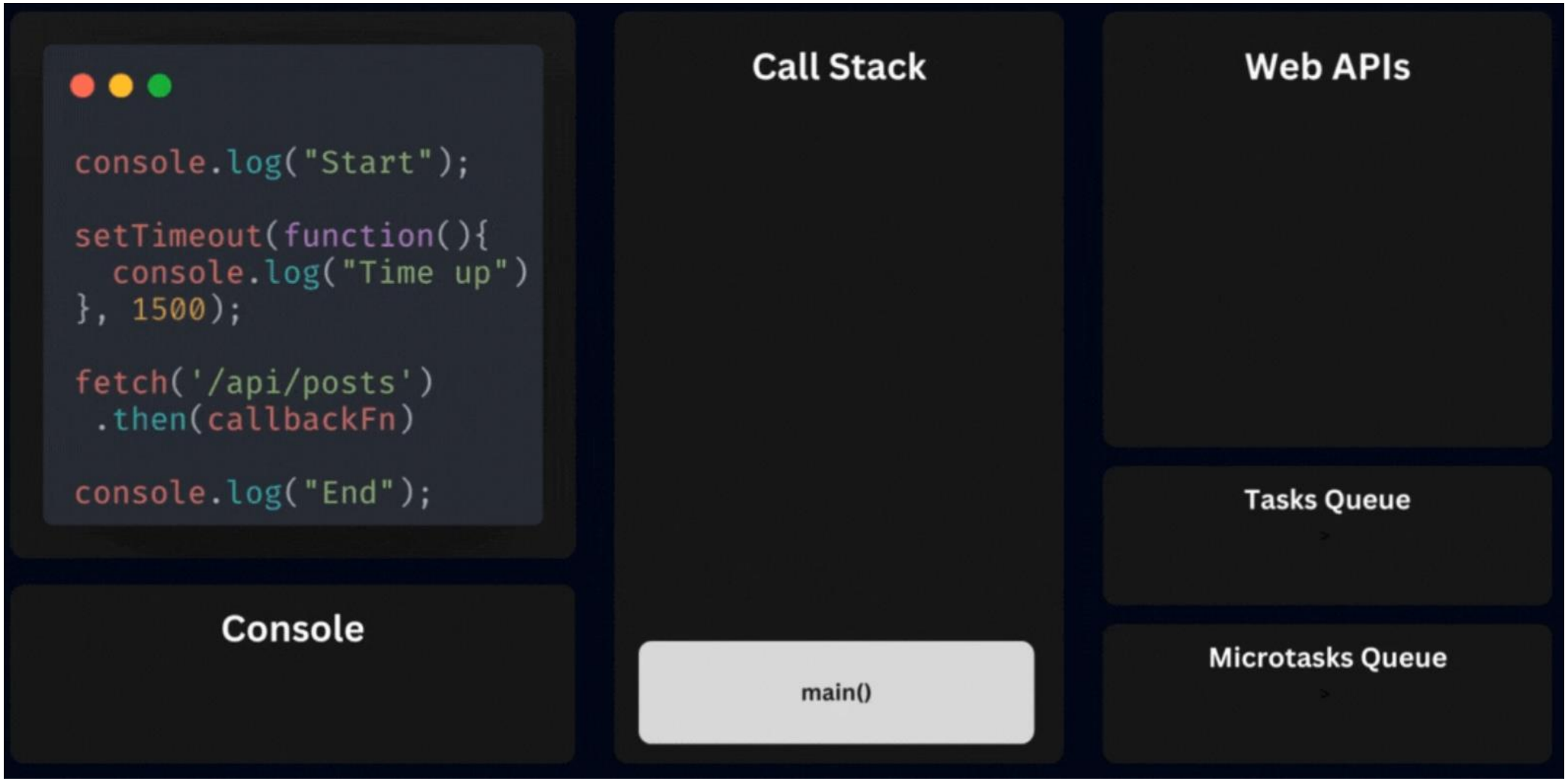


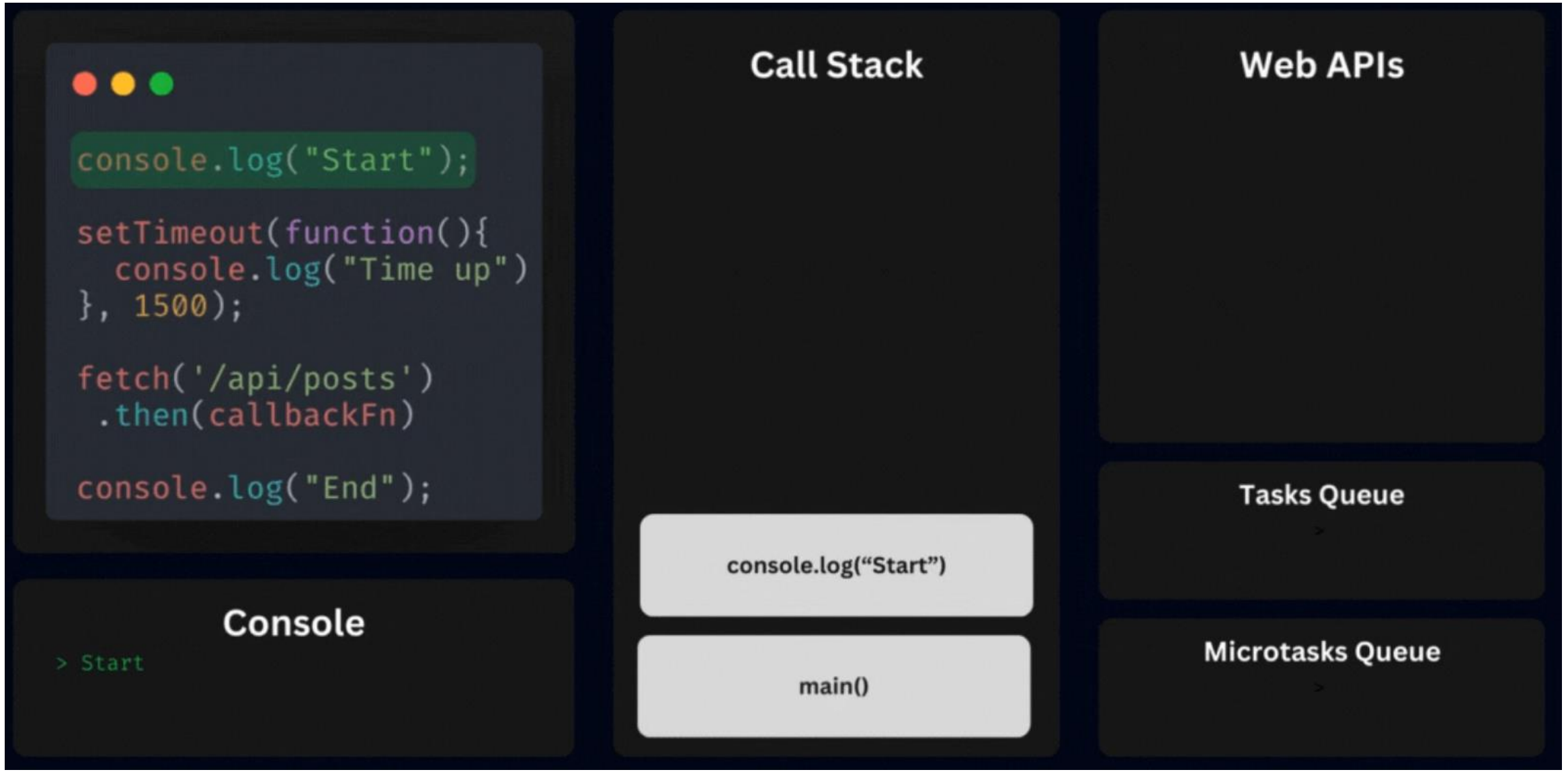
La cola Microtask tiene prioridad por sobre la Macrotask

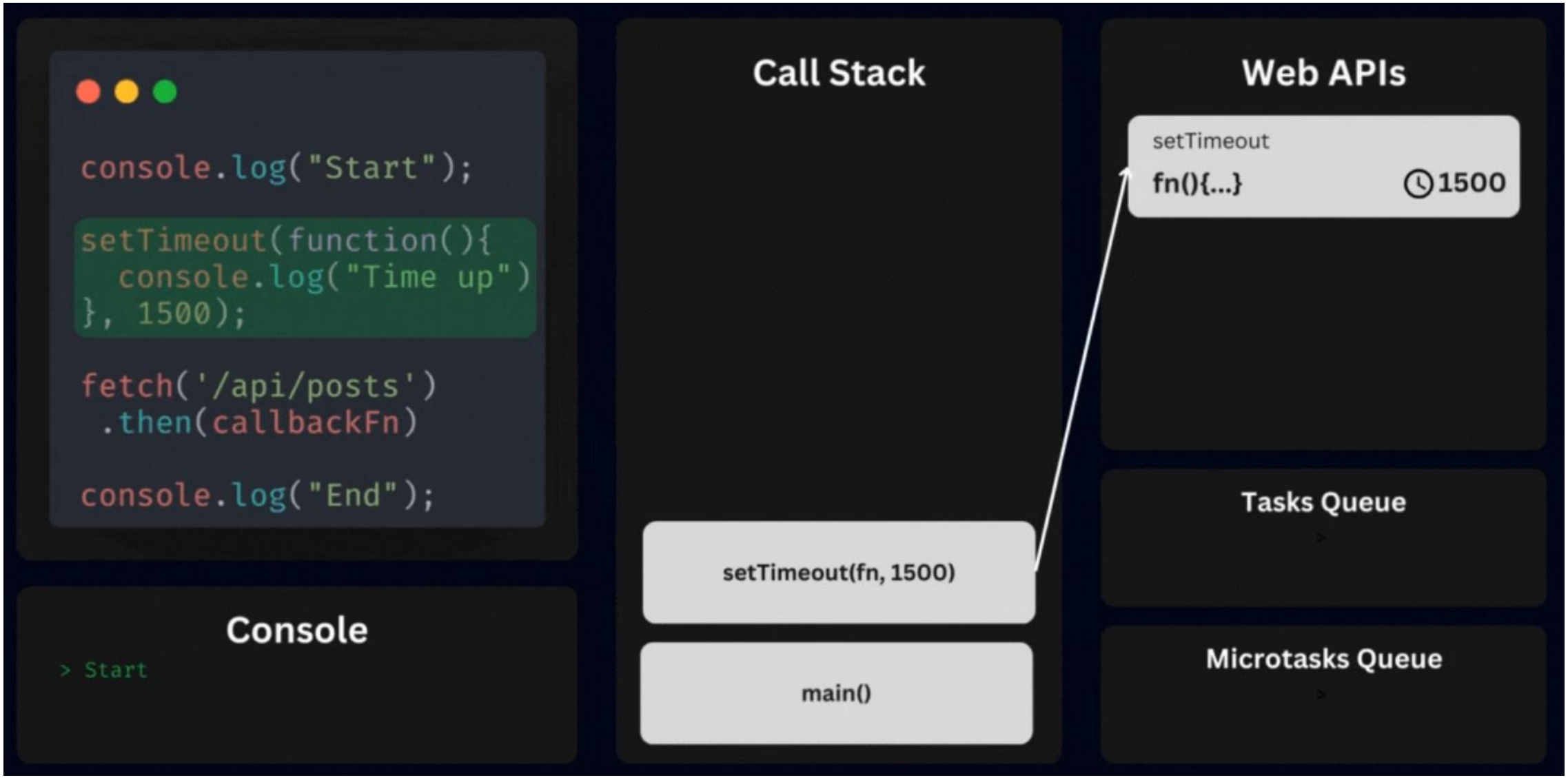


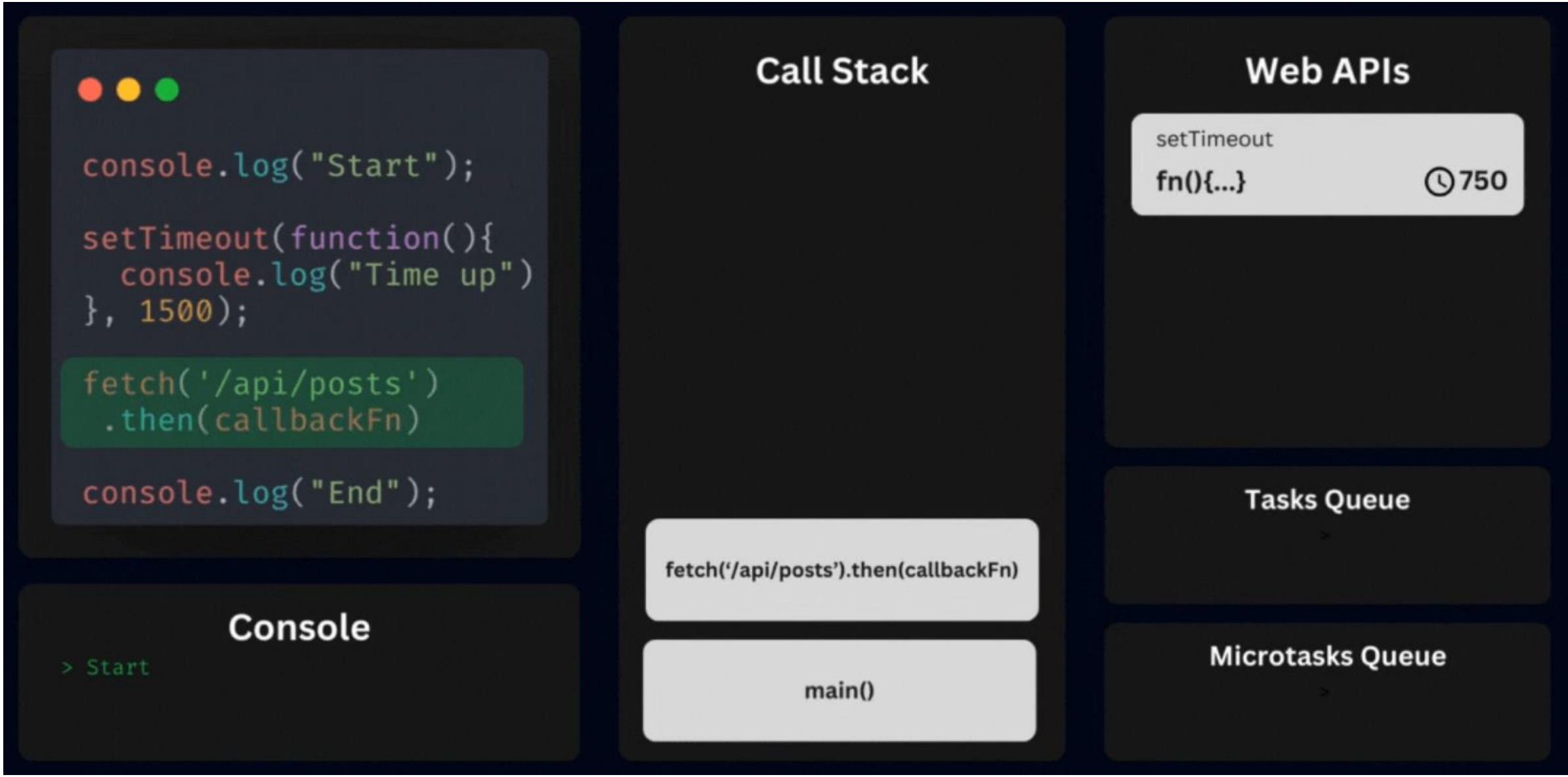
Ejemplo modelo paso a paso

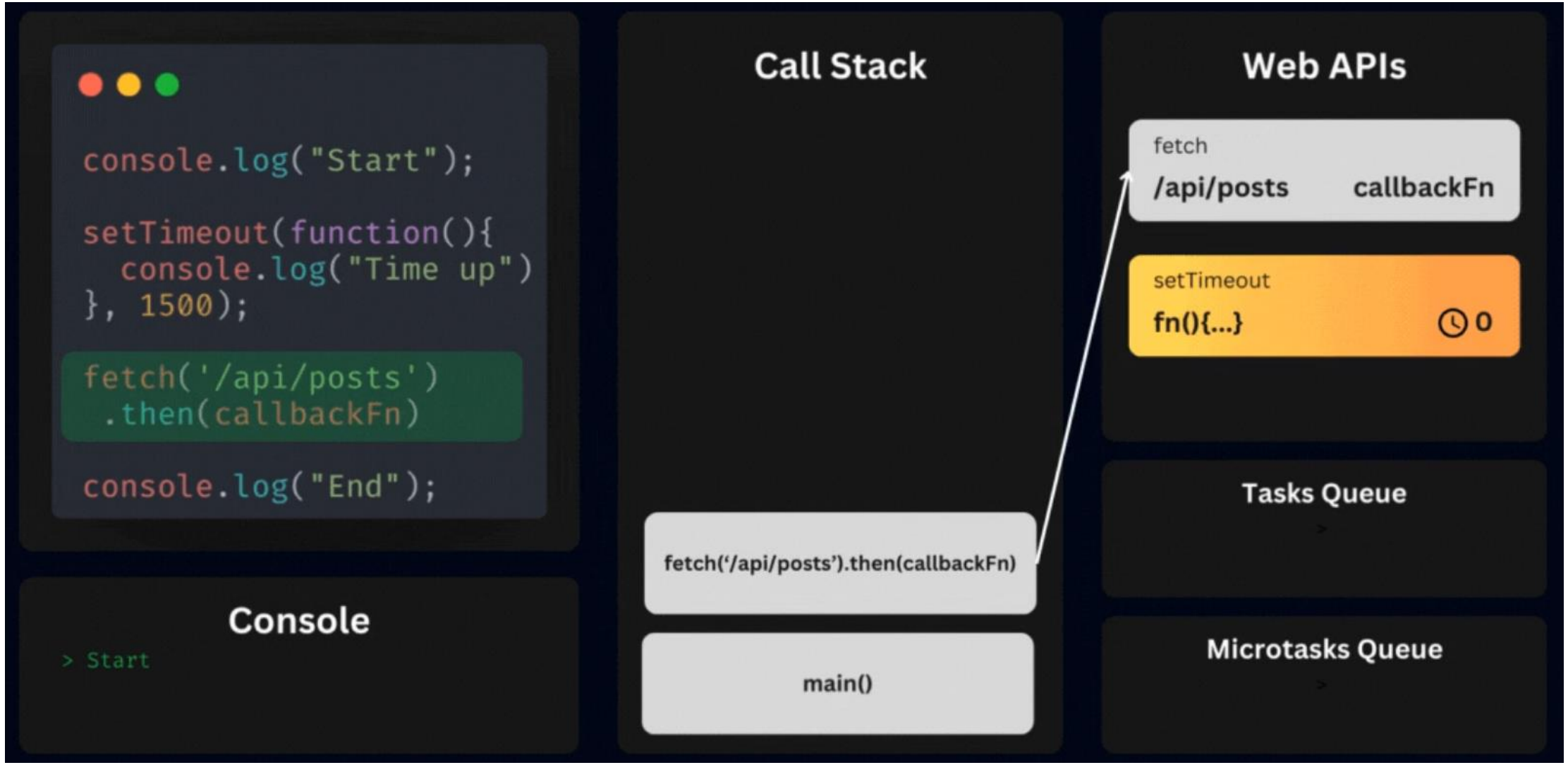


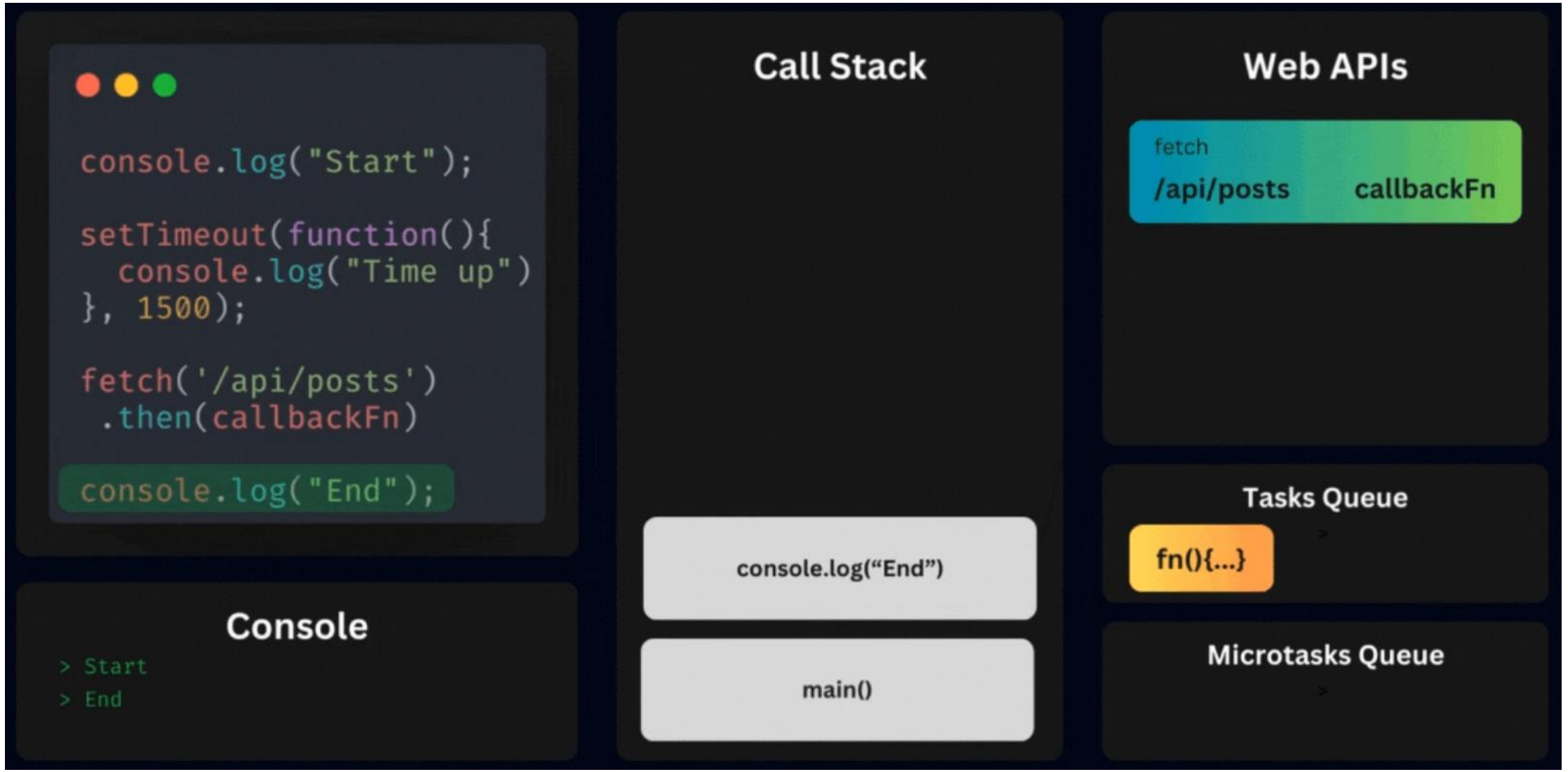

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
Call Stack
main()
Web APIs
Tasks Queue
Microtasks Queue

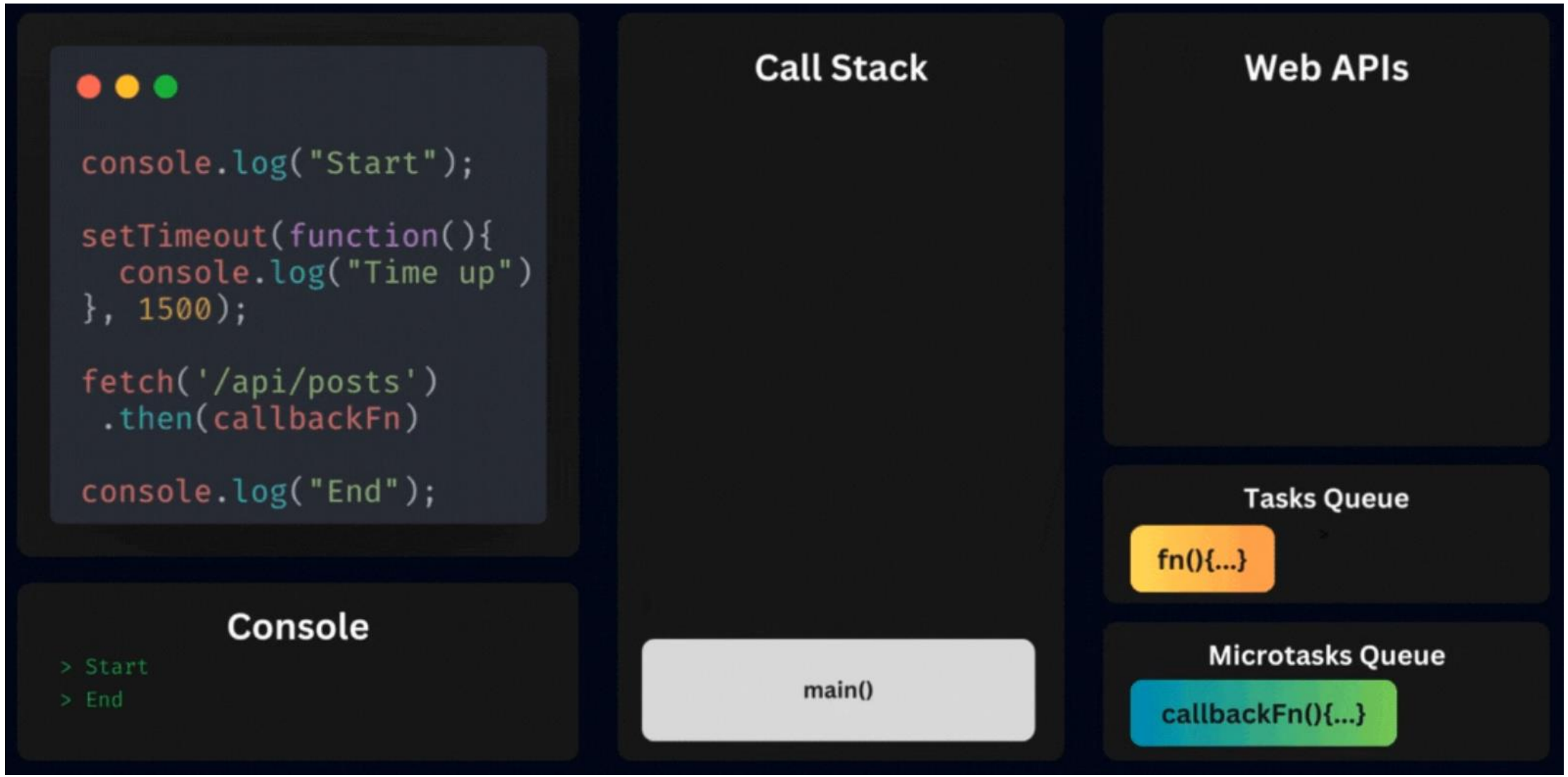

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
> Start
Call Stack
console.log("Start")
main()
Web APIs
Tasks Queue
Microtasks Queue

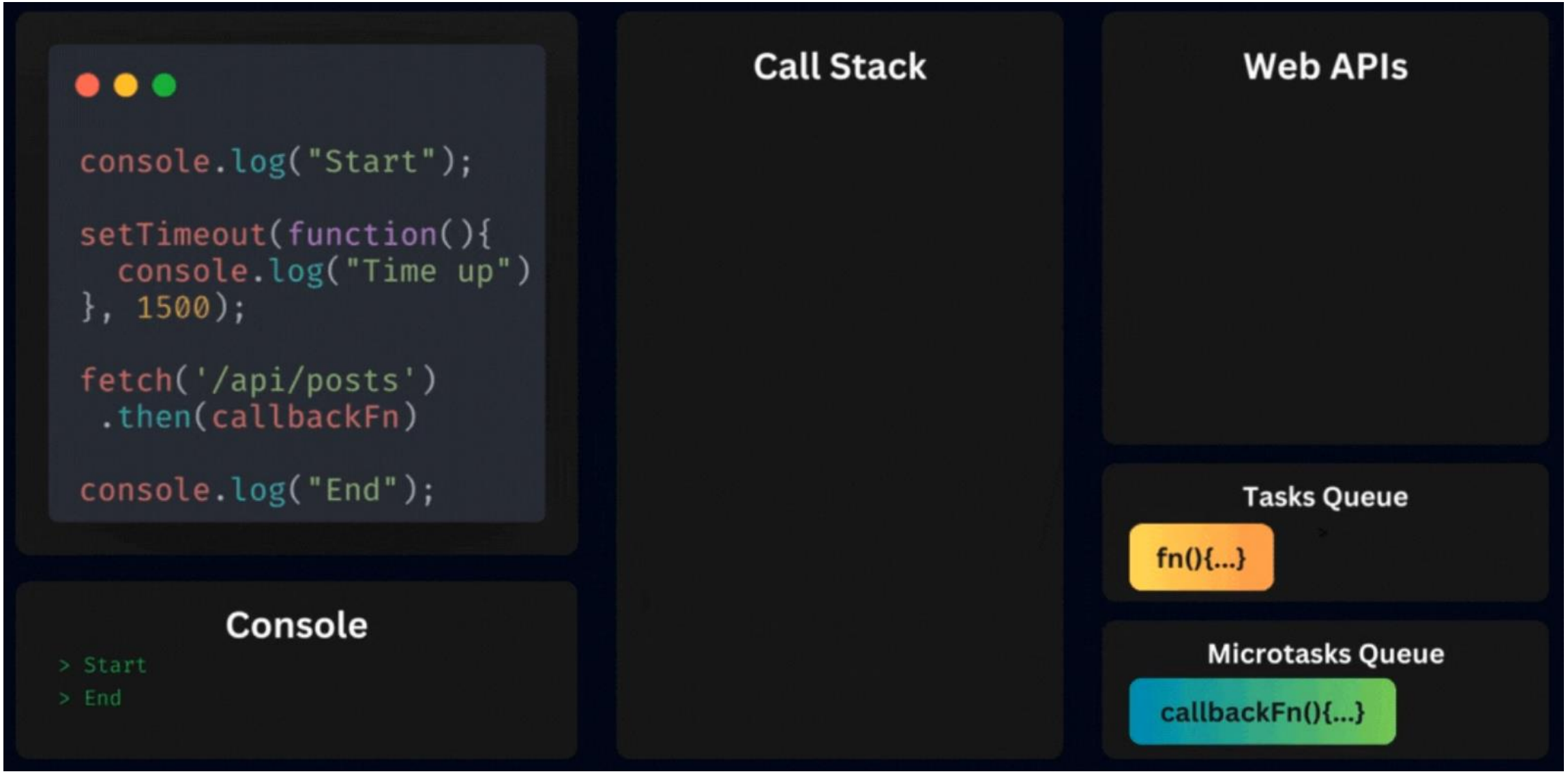

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
> Start
Call Stack
setTimeout(fn, 1500)
main()
Web APIs
setTimeout
fn(){...}
1500
Tasks Queue
Microtasks Queue

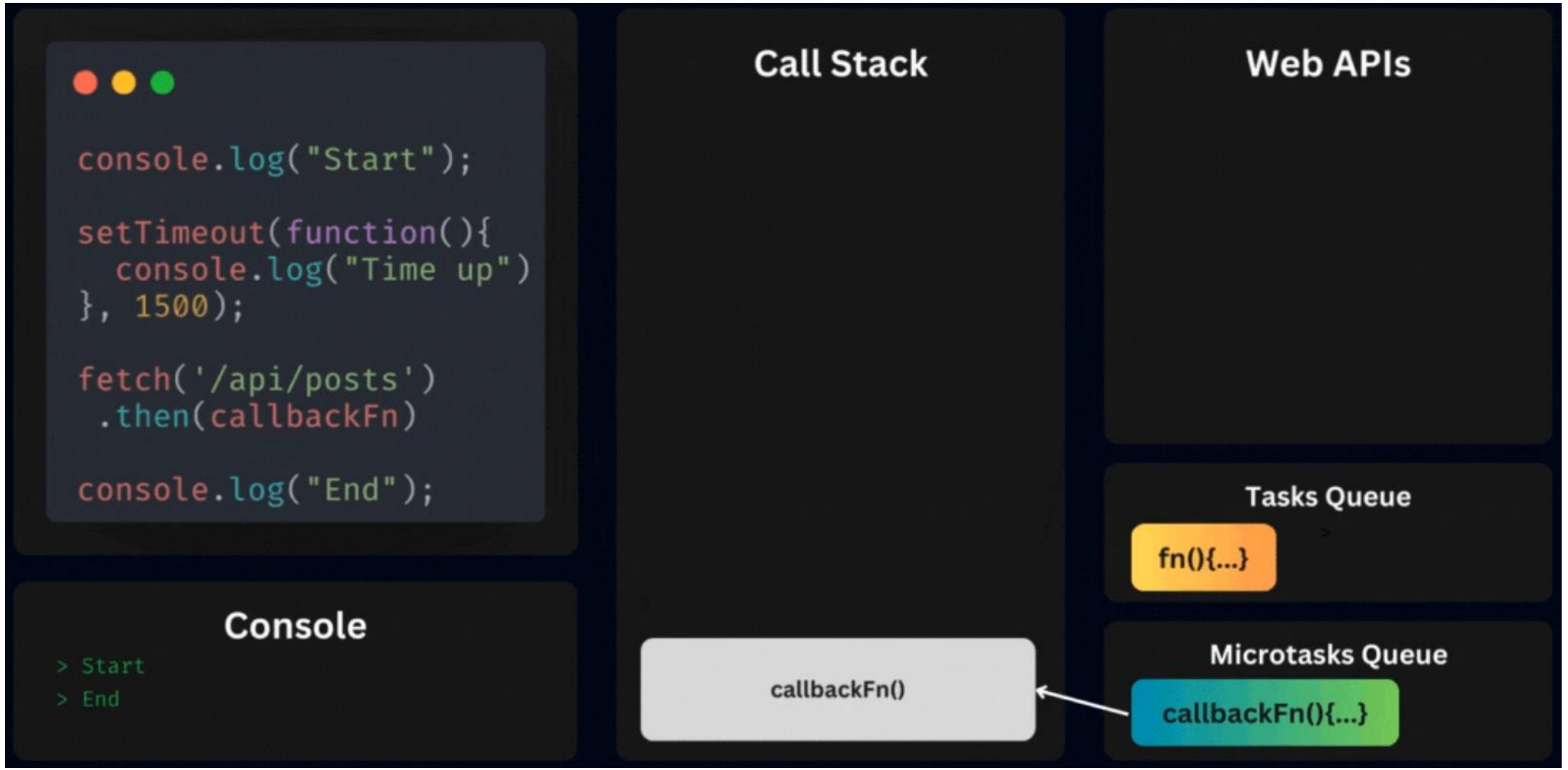

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
Web APIs
setTimeout
fn(){...}
750
Tasks Queue
Microtasks Queue
fetch('/api/posts').then(callbackFn)
main()
Console
> Start

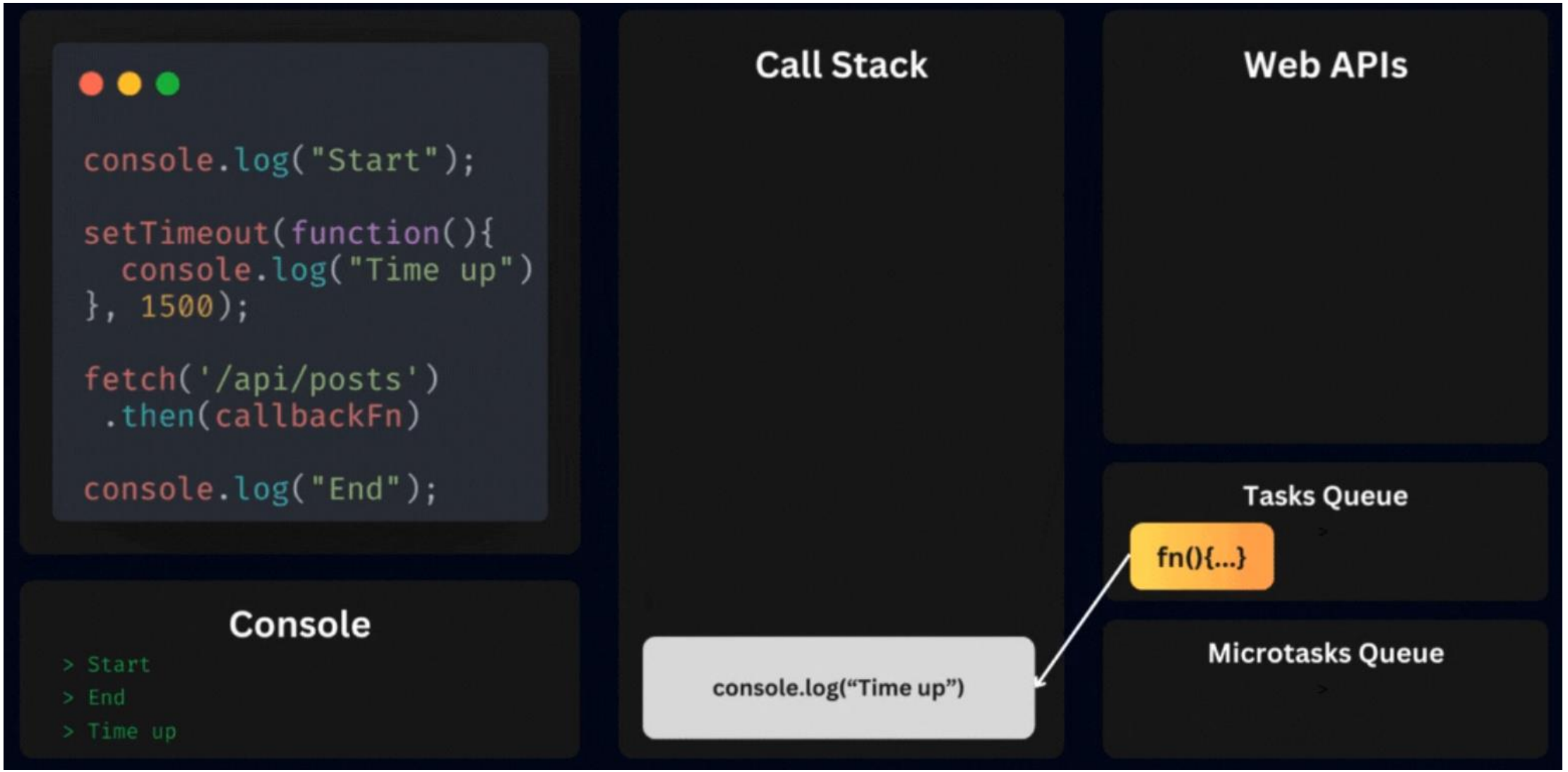

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
> Start
Call Stack
fetch('/api/posts').then(callbackFn)
main()
Web APIs
fetch
/api/posts
callbackFn
setTimeout
fn(){...}
0
Tasks Queue
Microtasks Queue

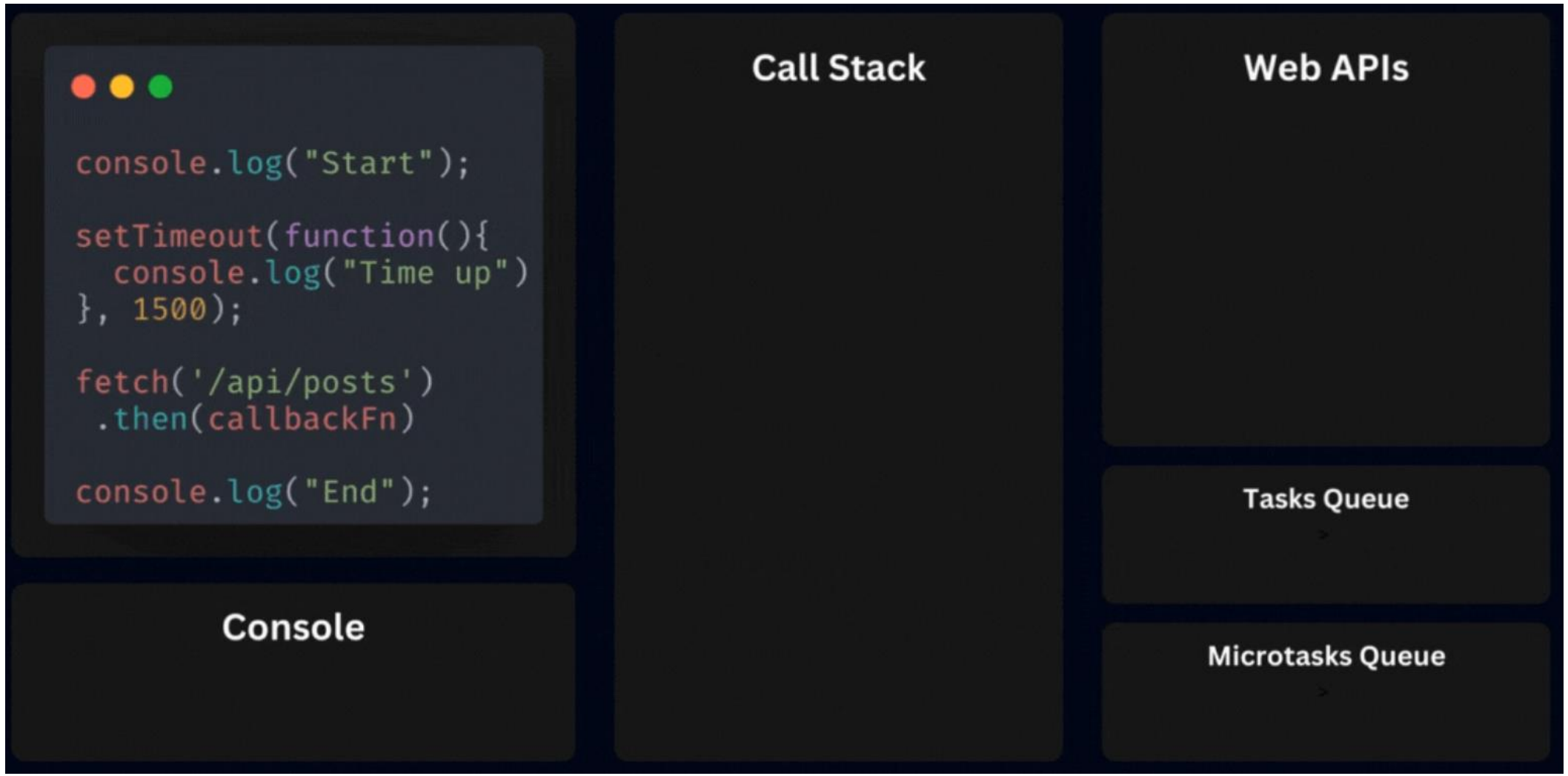

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
console.log("End")
main()
Web APIs
fetch
/api/posts
callbackFn
Tasks Queue
fn(){...}
Microtasks Queue
Console
> Start
> End

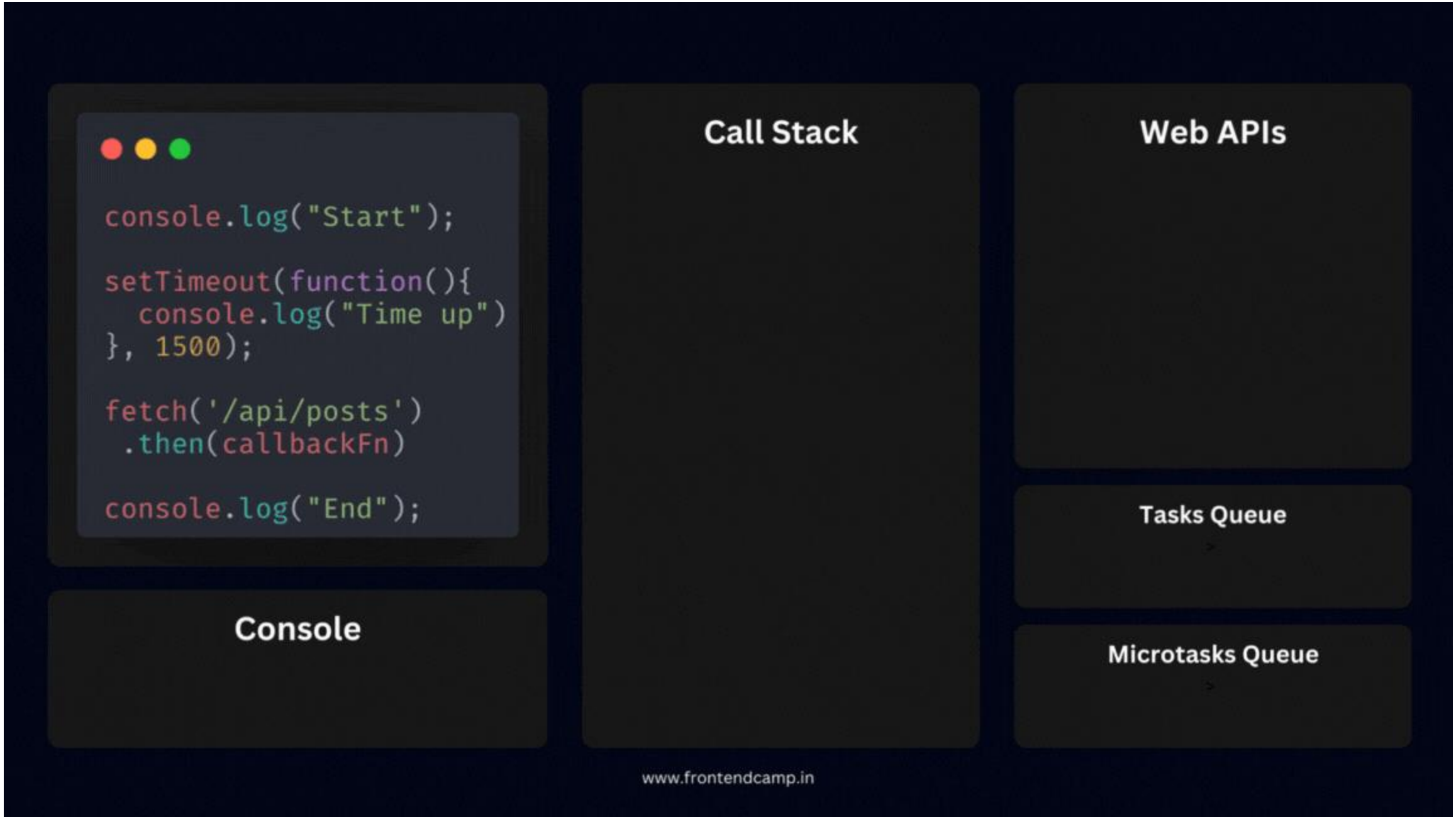

console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
Web APIs
Tasks Queue
fn(){...}
Console
> Start
> End
main()
Microtasks Queue
callbackFn(){...}


console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
Web APIs
Tasks Queue
fn(){...}
Microtasks Queue
callbackFn(){...}
Console
> Start
> End


console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
Web APIs
Tasks Queue
fn(){...}
Console
> Start
> End
callbackFn()
Microtasks Queue
callbackFn(){...}


console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Call Stack
Web APIs
Tasks Queue
fn(){...}
Microtasks Queue
console.log("Time up")
Console
> Start
> End
> Time up


console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
Call Stack
Web APIs
Tasks Queue
Microtasks Queue


console.log("Start");
setTimeout(function(){
console.log("Time up")
}, 1500);
fetch('/api/posts')
.then(callbackFn)
console.log("End");
Console
Call Stack
Web APIs
Tasks Queue
Microtasks Queue
www.frontendcamp.in

Componente	Qué hace	Tipo
Call Stack	Ejecuta el código, función por función	LIFO
Web APIs	Maneja tareas fuera del hilo JS (timers, fetch)	Externo
Macrotask Queue	Guarda callbacks listos para ejecutarse	FIFO, baja prioridad
Microtask Queue	Guarda callbacks de Promises	FIFO, alta prioridad
Event Loop	Mueve tareas de las colas al Call Stack	Monitoreo continuo

Comparación Síncrono vs. Asíncrono en JavaScript

Característica	Programación Síncrona	Programación Asíncrona
Flujo de Ejecución	Secuencial (una tarea tras otra)	Independiente/Concurrente (tareas pueden ejecutarse en paralelo)
Bloqueo del Hilo Principal	Sí (bloquea hasta que la tarea actual finaliza)	No (permite que el código continúe ejecutándose)
Capacidad de Respuesta de la UI	Puede congelar la interfaz de usuario durante tareas largas	Mantiene la interfaz de usuario responsiva
Complejidad	Más simple y predecible (inicialmente)	Más compleja (inicialmente, debido a la gestión de callbacks/promesas)
Casos de Uso Típicos	Tareas rápidas y predecibles (ej. cálculos básicos)	Peticiones de red, I/O de archivos, temporizadores, interacciones de usuario de larga duración

2. Callbacks: La Base de la Asincronía

Los callbacks representan el método original y fundamental para gestionar la asincronía en JavaScript. Su comprensión es esencial para apreciar la evolución de las técnicas asíncronas en el lenguaje.

2.1. ¿Qué son los Callbacks?

Una función **callback** es, en esencia, **una función que se pasa como argumento a otra función con la intención de que sea ejecutada en un momento posterior**, una vez que una operación específica ha sido completada.

El funcionamiento es análogo a la frase "te llamo de vuelta más tarde" (o en inglés, "I call you back"): **la función que recibe el callback lo invoca en el momento oportuno, generalmente después de que una tarea asíncrona ha finalizado.**

Es importante destacar que un callback no es inherentemente asíncrono; su asincronía depende directamente de la función que lo invoca. Un callback puede ser invocado de forma síncrona o asíncrona.

A continuación, se presentan ejemplos básicos para ilustrar el concepto:

- setTimeout (asíncrono):

```
console.log("Inicio");
setTimeout(function() {
  console.log("Este mensaje aparece después de 1 segundo (callback).");
}, 1000);
console.log("Fin");
// Salida: Inicio, Fin, Este mensaje... (después de 1 seg)
```

En este ejemplo, setTimeout es una Web API que programa la ejecución del callback después de un tiempo especificado, sin bloquear el hilo principal de JavaScript. El mensaje "Fin" se imprime antes del mensaje del callback.

- addEventListener (asíncrono, basado en eventos del DOM):

```
document.getElementById("miBoton").addEventListener("click", function() {
  console.log("¡Botón clickeado! (callback de evento)");
});
console.log("Esperando clicks...");
```

Aquí, el callback se ejecuta únicamente cuando el evento "click" sobre el elemento con ID "miBoton" ocurra. Esto demuestra cómo los callbacks son fundamentales para el manejo de eventos.

- Callback Síncrono (para ilustrar el concepto):

```
function procesarNumero(numero, operacion) {
  console.log("Procesando número:", numero);
  return operacion(numero);
}
function duplicar(x) {
  return x * 2;
}
let resultado = procesarNumero(5, duplicar);
console.log("Resultado:", resultado); // Salida: Procesando número: 5, Resultado: 10
```

En este caso, la función duplicar es un callback que se ejecuta de forma síncrona dentro de procesarNumero. Este ejemplo ayuda a comprender que la naturaleza asíncrona de un callback depende de la función que lo invoca, no del callback en sí.

2.2. Casos de Uso Típicos de Callbacks

Los callbacks son ampliamente utilizados en JavaScript para diversas tareas, especialmente aquellas que involucran operaciones asíncronas o manejo de eventos:

- **Operaciones Asíncronas**: Son fundamentales para manejar tareas como la lectura de archivos, la realización de peticiones HTTP (llamadas a APIs) y la implementación de temporizadores (como setTimeout y setInterval).
- **Manejo de Eventos:** Dada la naturaleza orientada a eventos de JavaScript, los callbacks son la base para responder a interacciones del usuario (clicks, pulsaciones de teclas) o a la carga de recursos.
- **Funciones Flexibles:** Permiten que una función ejecute diferentes comportamientos basados en una entrada, como en el caso de métodos de arrays como map o forEach, donde se pasa un callback para procesar cada elemento.

2.3. Problemas con los Callbacks: "Callback Hell" y Manejo de Errores

A pesar de su utilidad, el uso extensivo de callbacks, especialmente en secuencias de operaciones asíncronas dependientes unas de otras, dio origen a problemas significativos que llevaron al desarrollo de soluciones más avanzadas.

```
// La "Pirámide de la Perdición" 😱
hacerPaso1(function(resultado1) {
  hacerPaso2(resultado1, function(resultado2) {
    hacerPaso3(resultado2, function(resultado3) {
      hacerPaso4(resultado3, function(resultado4) {
        // cada nivel agrega indentación → difícil de leer y mantener
      });
    });
  });
});
```

- **"Callback Hell" (o "Pirámide de la Perdición"):**

Este problema surge cuando múltiples operaciones asíncronas deben ejecutarse secuencialmente, lo que resulta en una anidación profunda y progresiva de funciones de callback. El código resultante se vuelve extremadamente difícil de leer, mantener y depurar, a menudo esto es llamado "código espagueti"

Considérese el siguiente ejemplo que simula una secuencia de operaciones:

```
function obtenerUsuario(id, callback) {
  setTimeout(() => {
    console.log(`Usuario ${id} obtenido.`);
    callback({ id: id, nombre: "Alice" });
  }, 1000);
}

function obtenerPosts(usuario, callback) {
  setTimeout(() => {
    console.log(`Posts de ${usuario.nombre} obtenidos.`);
    callback(["Post 1", "Post 2"]);
  }, 1000);
}

function mostrarDatos(posts, callback) {
  setTimeout(() => {
    console.log("Mostrando posts:", posts);
    callback("Datos mostrados.");
  }, 1000);
}

// Ejemplo de "Callback Hell"
obtenerUsuario(123, function(usuario) {
  obtenerPosts(usuario, function(posts) {
    mostrarDatos(posts, function(mensaje) {
      console.log(mensaje);
    });
  });
});
// A medida que se añaden más pasos, la anidación se profundiza,
// dificultando el seguimiento del flujo de control.
```

Este patrón de anidación rompe la linealidad del código, haciendo que el flujo de control sea difícil de seguir y que el manejo de errores sea disperso y repetitivo. **Aunque los callbacks son fundamentales para el modelo de eventos de JavaScript y para la asincronía básica, su estructura de "paso de función" se convierte en una limitación arquitectónica cuando se busca componer y gestionar secuencias complejas de operaciones asíncronas.** No ofrecen un mecanismo inherente para representar el estado futuro de una operación o para encadenar resultados de forma limpia, lo que lleva a la necesidad de abstracciones superiores.

- **Manejo de Errores en Callbacks:**

El manejo de errores en cadenas de callbacks anidadas puede volverse notoriamente complicado y propenso a errores. Cada nivel de anidación requiere su propia lógica de manejo de errores, lo que a menudo conduce a código repetitivo y difícil de rastrear. La convención común para el manejo de errores en callbacks es el patrón *error-first*, donde el primer argumento del callback es un objeto de error (si existe) y el segundo es el resultado exitoso

Ejemplo manejo errores usando callbacks

```
function dividir(a, b, callback) {
  if (b === 0) {
    callback(new Error("No se puede dividir por cero"), null);
  } else {
    callback(null, a / b);
  }
}

dividir(10, 2, function(error, resultado) {
  if (error) {
    console.error("Error:", error.message);
  } else {
    console.log("Resultado:", resultado); // Salida: Resultado: 5
  }
});

dividir(10, 0, function(error, resultado) {
  if (error) {
    console.error("Error:", error.message); // Salida: Error: No se puede dividir por cero
  } else {
    console.log("Resultado:", resultado);
  }
});
```

En este ejemplo, cada callback debe verificar manualmente si se produjo un error, lo que se vuelve tedioso y propenso a omisiones en callbacks más profundas. Esta limitación estructural de los callbacks para la composición asíncrona evidenció la necesidad de un enfoque más robusto y manejable para la programación asíncrona.

3. Promises: Una Solución Estructurada

Las Promises (Promesas) surgieron como una mejora significativa sobre los callbacks, ofreciendo una forma más estructurada y robusta de manejar operaciones asíncronas y, crucialmente, de resolver el problema del "Callback Hell".

3.1. ¿Qué son las Promises?

Una Promise es un objeto que actúa como una "proxy" o un "placeholder" para un valor que no necesariamente se conoce en el momento en que se crea la Promise, pero que estará disponible en el futuro como resultado de una operación asíncrona. Permite asociar manejadores con el eventual éxito (valor resuelto) o el motivo del fallo (razón de rechazo) de una acción asíncrona.

Para entenderlo mejor, se puede usar la analogía de una promesa de un mozo en un restaurante. El mozo promete traer la comida. Mientras se espera, se pueden realizar otras actividades. La promesa se "cumple" cuando el mozo trae la comida, o se "rechaza" si hay un problema (por ejemplo, el plato no está disponible en ese día).

Las **Promises transforman la asincronía de un modelo basado en "eventos y funciones sueltas" a un modelo basado en "objetos con estado y un contrato claro"**. Esto no solo mejora la legibilidad y el manejo de errores, sino que también proporciona una base más sólida y predecible para construir lógica asíncrona compleja, lo cual es vital para las aplicaciones web modernas que dependen en gran medida de interacciones de red y de usuario.

Las ventajas de las Promises sobre los callbacks son notables: ofrecen una manera más organizada y legible de manejar operaciones asíncronas, evitan la anidación excesiva característica del "Callback Hell" y proporcionan un mecanismo de manejo de errores más robusto y centralizado.

3.2. Estados de una Promise

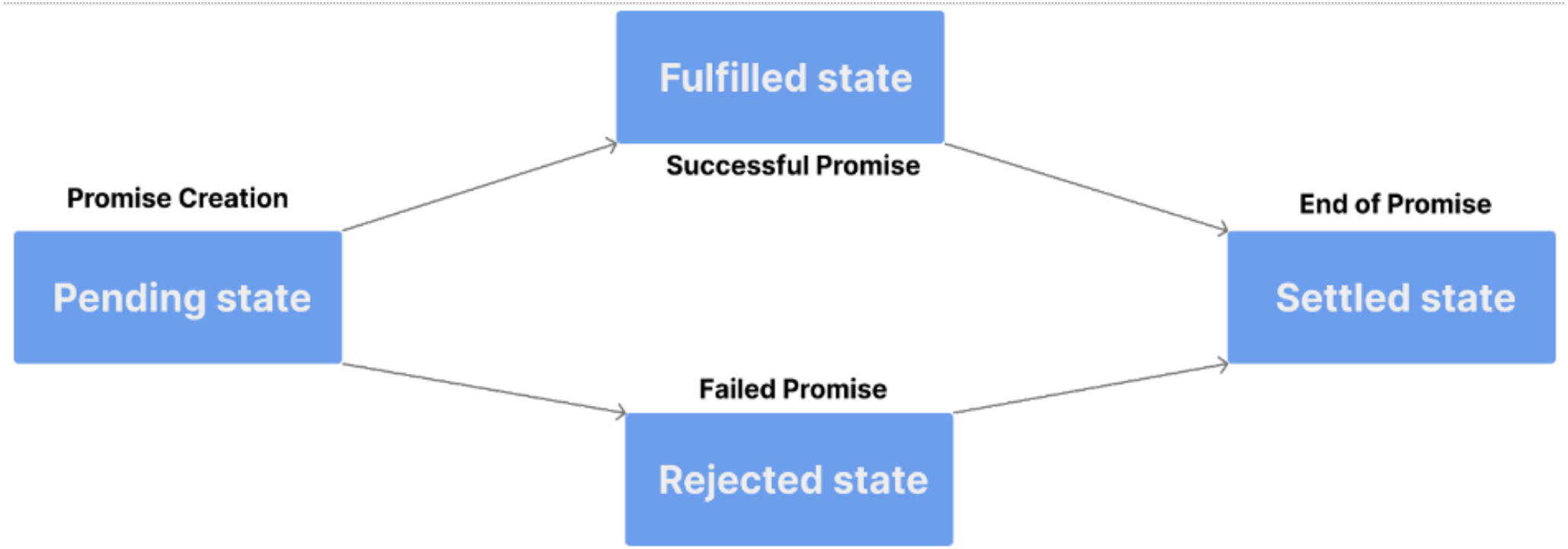
Una Promise, a lo largo de su ciclo de vida, puede encontrarse en uno de los tres estados principales :

- **pending (Pendiente)**: Este es el estado inicial de una Promise. *Indica que la operación asíncrona asociada aún no se ha completado ni ha fallado*. La Promise permanece en este estado mientras el resultado de la operación es desconocido.
- **fulfilled (Cumplida / Resuelta):** Una Promise entra en este estado cuando la *operación asíncrona se ha completado con éxito*. En este punto, la Promise tiene un valor resultante disponible.
- **rejected (Rechazada):** Este estado *indica que la operación asíncrona ha fallado*. La Promise contiene una razón para la falla, que generalmente es un objeto de error.

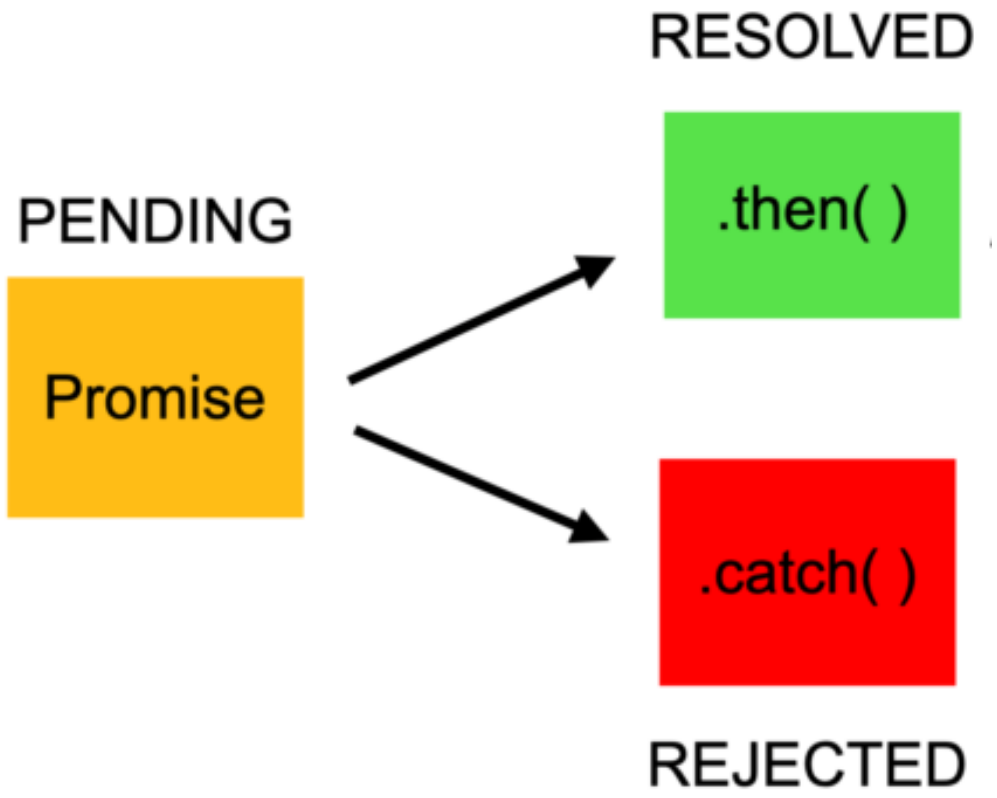
- **settled (Asentada):** Una Promise se considera settled cuando ya no está en estado pending; es decir, ha sido fulfilled o rejected. Este es el estado final de una Promise, y una vez que una Promise está settled, su estado no puede cambiar.

Las Promises, al encapsular el estado de una operación asíncrona, permiten adjuntar manejadores (*.then(), .catch(), .finally()*) en cualquier momento, incluso si la Promise ya se ha resuelto. Este modelo basado en objetos con estado y un contrato claro es fundamental para la robustez de las aplicaciones modernas.

Ilustración de estados



Métodos para consumir una Promesa



3.3. Creación de Promises

Para comprender a fondo el funcionamiento de las Promises, es fundamental que aprendan a crearlas desde cero. Una Promise se construye utilizando el constructor *new Promise()*, el cual recibe una función ejecutora como argumento. Esta función ejecutora, a su vez, toma dos parámetros: ***resolve*** **y** ***reject***. Estas **son funciones de callback que se utilizan para cambiar el estado de la Promise.**

- **resolve(value):** Se invoca cuando la operación asíncrona se completa con éxito. El *value* pasado a *resolve* será el valor con el que la Promise se resuelve (fulfilled).
- **reject(reason):** Se llama cuando la operación asíncrona falla. La *reason* (generalmente un objeto Error) será la razón con la que la Promise se rechaza (rejected).

La función ejecutora se ejecuta inmediatamente de forma síncrona cuando se crea la Promise.

Ejemplo Básico: Una Promise que se resuelve después de un tiempo

Este es el ejemplo más fundamental para entender la creación de una Promise. Simula una operación asíncrona que tarda un tiempo en completarse.

```
const miPrimeraPromesa = new Promise((resolve, reject) => {
  // Simulamos una operación asíncrona con setTimeout
  setTimeout(() => {
    const exito = true; // Podría ser el resultado de una operación real (ej. fetch exitoso)

    if (exito) {
      resolve("¡Datos obtenidos con éxito!"); // La promesa se resuelve con un valor
    } else {
      reject(new Error("Error al obtener los datos.")); // La promesa se rechaza con un error
    }
  }, 2000); // La operación tarda 2 segundos
});

// Consumimos la promesa
miPrimeraPromesa
 .then(mensaje => {
    console.log(mensaje); // Se ejecuta si la promesa se resuelve
  })
 .catch(error => {
    console.error(error.message); // Se ejecuta si la promesa se rechaza
  });

console.log("Solicitud de datos enviada. Esperando respuesta...");
// Salida esperada:
// Solicitud de datos enviada. Esperando respuesta...
// (después de 2 segundos)
// ¡Datos obtenidos con éxito!
```

En este ejemplo, *miPrimeraPromesa* se crea y entra en estado *pending*. El *setTimeout* simula el trabajo asíncrono. Después de 2 segundos, si *exito* es *true*, se llama a *resolve()*, cambiando el estado de la promesa a *fulfilled* y activando el .*then()*. Si *exito* fuera *false*, se llamaría a *reject()*, cambiando el estado a *rejected* y activando el .*catch()*

Otro ejemplo: Promise con lógica condicional

Las Promises pueden encapsular lógica más compleja, donde el resultado (resolved o rejected) depende de condiciones internas.

```
function verificarNumeroAleatorio() {
  return new Promise((resolve, reject) => {
    const numeroAleatorio = Math.random(); // Genera un número entre 0 (inclusive) y 1 (exclusive)
    console.log(`Número aleatorio generado: ${numeroAleatorio}`);

    if (numeroAleatorio >= 0.5) {
      resolve("¡El número es mayor o igual a 0.5! Éxito."); // La promesa se resuelve
    } else {
      reject(new Error("El número es menor a 0.5. Fallo.")); // La promesa se rechaza
    }
  });
}

// Consumimos la promesa
verificarNumeroAleatorio()
 .then(resultado => {
    console.log(resultado);
  })
 .catch(error => {
    console.error(`Error: ${error.message}`);
  });
```

Promises Inmediatas: Promise.resolve() y Promise.reject()

A veces, se necesita crear una Promise que ya esté resuelta o rechazada de inmediato.

- Promise.resolve(value): Crea una Promise que se resuelve inmediatamente con el *value* dado.

```
const promesaExitosa = Promise.resolve("¡Siempre exitosa!");
promesaExitosa.then(mensaje => console.log(mensaje)); // Salida: ¡Siempre exitosa!
```

- Promise.reject(reason): Crea una Promise que se rechaza inmediatamente con la *reason* dada.

```
const promesaFallida = Promise.reject(new Error("¡Siempre fallida!"));
promesaFallida.catch(error => console.error(error.message)); // Salida: ¡Siempre fallida!
```

Comprender la creación de promesas es el primer paso para dominar la programación asíncrona moderna en JavaScript, ya que sienta las bases para el uso de *.then()*, *.catch()*, y eventualmente *async/await*.

3.3. Consumiendo Promises: .then(), .catch(), .finally()

Las Promises se consumen adjuntando funciones de callback a sus métodos *.then(), .catch() y .finally()*. Estos métodos permiten definir qué acciones se deben tomar una vez que la Promise posea cierto estado.

- **.then(onFulfilled):** Este método es invocado cuando la Promise toma el estado *fulfilled*. El primer argumento, *onFulfilled*, es una función de callback que recibe el valor resuelto de la Promise.
- **.catch(onRejected):** Este método se utiliza específicamente para manejar los casos en que una Promise toma el estado *rejected*. El callback *onRejected* recibe la razón del error. Es el método más recomendado para gestionar errores en una cadena de Promises, ya que centraliza el manejo de excepciones.

- **.finally(onFinally):** El método finally() ejecuta su callback onFinally siempre, sin importar si la Promise es fulfilled o rejected. Es particularmente útil para realizar operaciones de limpieza o acciones finales que deben ocurrir independientemente del resultado de la Promise, como ocultar un spinner de carga.

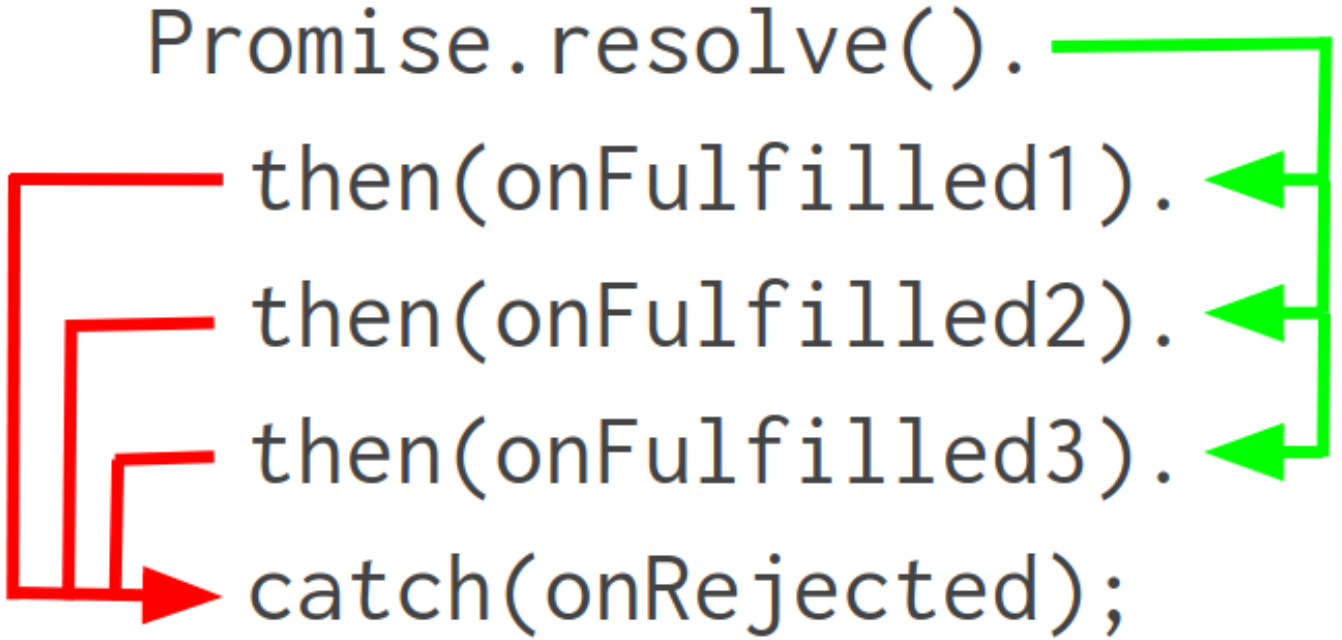
A continuación, se presenta un ejemplo práctico consumiendo una Promesa.

```javascript
// Creamos una función que retorna una Promise que se resuelve luego de cierto tiempo
function esperar(ms) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`Pasaron ${ms} milisegundos`);
    }, ms);
  });
}

// Consumimos la Promise
esperar(2000)
  .then(mensaje => {
    console.log(mensaje); // "Pasaron 2000 milisegundos"
  })
  .catch(error => {
    console.error('Ocurrió un error:', error.message);
  })
  .finally(() => {
    console.log('La espera ha finalizado.');
  });
```

3.4. Encadenamiento de Promises (Promise Chaining)

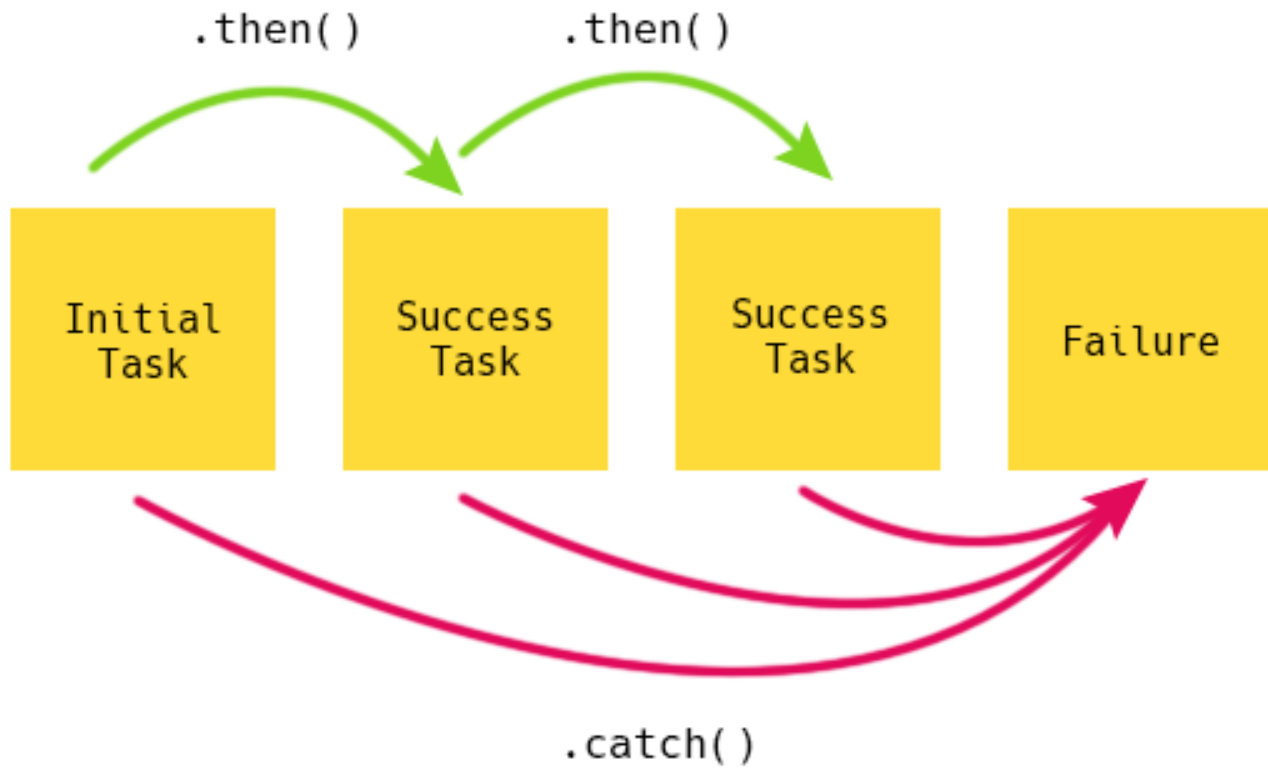
La capacidad de encadenar *Promises es* una de sus características más potentes y la principal solución al "Callback Hell". Cada método *.then()* (*y .catch()*) devuelven una nueva Promise. Esto permite ejecutar operaciones asíncronas de manera secuencial y legible, manteniendo un flujo de código más plano y fácil de seg...



Cadena de promesas con fetch

```
const url = "https://jsonplaceholder.typicode.com/posts/1"; // Una API de prueba

fetch(url) // fetch() retorna una Promise
 .then(response => {
    // El primer.then() se ejecuta cuando la respuesta HTTP es recibida.
    // Se verifica si la respuesta es exitosa (status 2xx).
    if (!response.ok) {
      throw new Error(`Error HTTP: ${response.status}`); // Se lanza un error para manejar respuestas no exitosas
    }
    return response.json(); // response.json() también retorna una Promise, para parsear el cuerpo de la respuesta como JSO
 })
 .then(data => {
    // El segundo.then() se ejecuta cuando el JSON ha sido parseado exitosamente.
    console.log("Datos obtenidos:", data);
 })
 .catch(error => {
    //.catch() maneja cualquier error que ocurra en la cadena anterior (problemas de red, errores HTTP, errores de parseo J
    console.error("Hubo un problema con la petición Fetch:", error);
 })
 .finally(() => {
    //.finally() se ejecuta siempre, al final del ciclo de la Promise, independientemente del éxito o fracaso.
    console.log("Petición Fetch finalizada.");
 });
```


.then()
.then()
Initial Task
Success Task
Success Task
Failure
.catch()

3.5. Manejo de Errores en Promises

Propagación de Errores: Cuando una Promise pasa al estado *rejected* en cualquier punto en una cadena de Promises, el control salta automáticamente al *.catch()*. Esto centraliza el manejo de errores, eliminando la necesidad de verificar y manejar errores en cada paso individual, como ocurría con la convención *error-first* de los callbacks.

¿Cuándo usar Promises vs Async/Await?

Tanto Promises como async/await resuelven el mismo problema. La diferencia es de estilo y legibilidad:

Situación	Recomendación
Operación asíncrona simple	Cualquiera de los dos
Múltiples operaciones secuenciales	async/await (más legible)
Operaciones en paralelo	Promise.all() (ver abajo)
Manejo de errores centralizado	async/await con try/catch
Código legacy o librerías antiguas	Callbacks o Promises

Promise.all() — Operaciones en paralelo

Este es un método que nos permite realizar consultas de forma paralela.

Promise.all() recibe un array de Promises y espera a que todas se resuelvan. Si alguna falla, rechaza inmediatamente.

```
// ❌ Secuencial: lento (espera una para empezar la otra)
const usuario = await obtenerUsuario(1);   // 1 segundo
const posts   = await obtenerPosts(1);      // 1 segundo
// Total: ~2 segundos

// ✅ En paralelo con Promise.all: más rápido
const [usuario, posts] = await Promise.all([
  obtenerUsuario(1),   // se inician al mismo tiempo
  obtenerPosts(1),
]);
// Total: ~1 segundo (el tiempo del más lento)
```

La performance de nuestra aplicación aumenta considerablemente al utilizar este método.

4. Async/Await: Simplificando las Promises

async/await representa la evolución más reciente y ergonómica en el manejo de la asincronía en JavaScript. Introducido en ECMAScript 2017, es una capa de "azúcar sintáctico" (syntactic sugar) construida sobre las Promises, que permite escribir código asíncrono que se ve y se comporta de manera muy similar al código síncrono.

4.1. ¿Qué es async/await?

La principal ventaja de async/await radica en su capacidad para mejorar drásticamente la legibilidad y la mantenibilidad del código asíncrono. Al permitir que el código asíncrono se escriba en un estilo más lineal, se simplifica su comprensión y seguimiento.

```
// Con Promises (legible pero verboso) 🤔
obtenerUsuario()
  .then(usuario => obtenerPosts(usuario.id))
  .then(posts => mostrar(posts))
  .catch(err => console.error(err));

// Con async/await (limpio y lineal) ✅
async function cargarDatos() {
  try {
    const usuario = await obtenerUsuario();
    const posts = await obtenerPosts(usuario.id);
    mostrar(posts);
  } catch (err) {
    console.error(err);
  }
}
```

4.2. La palabra clave async

La palabra clave *async* se utiliza para declarar una función como asíncrona.

El comportamiento de una función *async* es el siguiente:

- Una función async siempre retorna una Promise.
- Si la función async retorna un valor que no es una Promise (por ejemplo, una cadena, un número o un objeto), JavaScript lo envuelve automáticamente en una Promise resuelta (Promise.resolve()).
- El uso de async es un requisito indispensable para poder utilizar la palabra clave await dentro de la función.

Ejemplo

```
async function saludar() {
  return "Hola Mundo";
}
saludar().then(mensaje => console.log(mensaje)); // Salida: Hola Mundo (envuelto en Promise)

async function miFuncionAsincrona() {
  // Aquí se puede usar await
}
```

4.3. La palabra clave await

La palabra clave *await* solo puede ser utilizada dentro de una función *async*. Su propósito es "pausar" la ejecución de la función async hasta que la Promise a la que se aplica pase al estado settled (es decir, que la promesa tomó el estado fulfilled o rejected).

Cuando la Promise sea fulfilled, *await* retorna el valor resuelto de esa Promise. Si la Promise es rejected, await lanza un error, que puede (o no) ser capturado por un bloque try...catch.

Ejemplo con fetch API (usando async/await):

```javascript
async function obtenerDatosDeUsuario(id) {
  try {
    console.log("Iniciando petición de datos de usuario...");
    // await pausa la ejecución hasta que la Promise de fetch se resuelva
    const response = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    if (!response.ok) {
      throw new Error(`Error al obtener usuario: ${response.status}`);
    }
    // await pausa la ejecución hasta que la Promise de response.json() se resuelva
    const usuario = await response.json();
    console.log("Usuario obtenido:", usuario.name);

    console.log("Iniciando petición de posts del usuario...");
    const postsResponse = await fetch(`https://jsonplaceholder.typicode.com/posts?userId=${id}`);
    if (!postsResponse.ok) {
      throw new Error(`Error al obtener posts: ${postsResponse.status}`);
    }
    const posts = await postsResponse.json();
    console.log(`Número de posts de ${usuario.name}: ${posts.length}`);

    return { usuario, posts };
  } catch (error) {
    console.error("Ocurrió un error en obtenerDatosDeUsuario:", error.message);
    // Se puede re-lanzar el error para que sea manejado por el llamador, o devolver un valor por defecto
    throw error;
  } finally {
    console.log("Función obtenerDatosDeUsuario finalizada.");
  }
}

// Llamada a la función async
obtenerDatosDeUsuario(1)
 .then(data => console.log("Proceso completo:", data))
 .catch(err => console.error("Error final del proceso:", err.message));

console.log("Este mensaje se imprime antes de que la función async termine su ejecución interna.");
```

En este ejemplo, el uso de await hace que el código dentro de *obtenerDatosDeUsuario* se lea como si fuera síncrono, a pesar de que las operaciones *fetch()* y *json()* son inherentemente asíncronas. El bloque *try…catch* maneja los errores de manera centralizada, capturando cualquier rechazo de alguna Promise o excepciones lanzadas dentro del bloque *try*.

4.4. Manejo de Errores con try...catch en async/await

Una de las mayores ventajas de *async/await* es la simplicidad con la que se pueden manejar los errores, asemejándose al manejo de errores en el código síncrono mediante los bloques *try...catch*.

Cualquier error que ocurra dentro del bloque *try* (incluyendo las Promises que sean rechazadas por una operación await) será capturado por el bloque catch asociado. Esto proporciona un mecanismo de manejo de errores familiar y robusto.

El ejemplo anterior de *obtenerDatosDeUsuario* ilustra este punto: si *fetch* falla debido a un problema de red, o si *response.json()* lanza un error durante el parseo, la ejecución dentro de la función *async* se detendrá y el control pasará directamente al bloque *catch*.

4.5. Flujo de Ejecución de async/await

El flujo de ejecución de *async/await* es una abstracción inteligente sobre el modelo de Promises y el Event Loop.

1. Cuando se invoca una función *async*, esta comienza a ejecutarse de forma síncrona hasta que encuentra la primera expresión *await*.
2. Al encontrar *await*, la ejecución de la *función async* se "pausa".
3. En este punto, el control se devuelve al código que llamó a la función async (o al Event Loop si no hay más código síncrono pendiente). Esto permite que el hilo principal de JavaScript pueda seguir ejecutando otras tareas y no se bloquee.

4. La promesa que se está esperando (await) se va a resolver (o rechazar) en segundo plano, esto es gestionado por las Web APIs y las colas de macrotareas/microtareas.
5. Una vez que la promise se settled (se fulfilled o se rejected), la ejecución de la función async se reanuda desde el punto donde se detuvo. Si la Promise se fulfilled, await devuelve el valor resuelto. Si se rejected, await lanza una excepción, que puede ser capturada por un try…catch.

Errores comunes con async/await

```
// ❌ Error 1: olvidar await (la variable recibe una Promise, no el valor)
async function cargarUsuario() {
  const usuario = obtenerUsuario(); // falta await!
  console.log(usuario); // Promise { <pending> }
}

// ❌ Error 2: usar await fuera de una función async
const datos = await fetch("..."); // SyntaxError

// ❌ Error 3: no manejar errores (Promise rechazada sin catch)
async function cargar() {
  const datos = await fetch("url-que-falla"); // si falla, explota silencioso
}

// ✅ Correcto
async function cargar() {
  try {
    const datos = await fetch("url-que-falla");
    return datos;
  } catch (err) {
    console.error("Error al cargar:", err.message);
  }
}
```

Conclusión final

Callbacks

→ Legibles para casos simples

→ ❌ Callback Hell en flujos complejos

Promises

→ Encadenamiento limpio con .then()

→ ❌ Cadenas largas siguen siendo verbosas

Async/Await

→ Código asíncrono que parece síncrono

→ ✅ Estándar actual en la industria

La utilización de Callbacks asíncronas, Promesas y async-await son herramientas que los desarrolladores disponemos para gestionar código asíncrono no bloqueante.